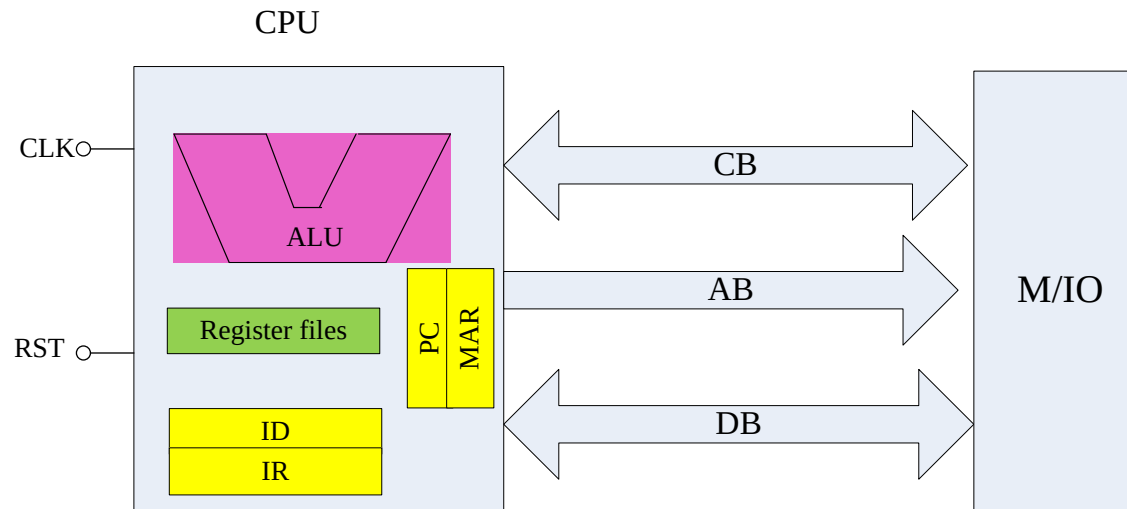


# Review for Chapter 1

- 微型计算机组成结构和基本工作原理
  - 组成结构
    - CPU (运算器, 控制器, 寄存器), 存储器, input/output 接口, 系统总线
  - 基本工作原理
    - 时钟周期, 指令周期, 指令助记符, 机器码, 指令系统, 指令执行过程 (取指令, PC+1, 译码, 执行), CPU 读/写



# Review for Chapter 1

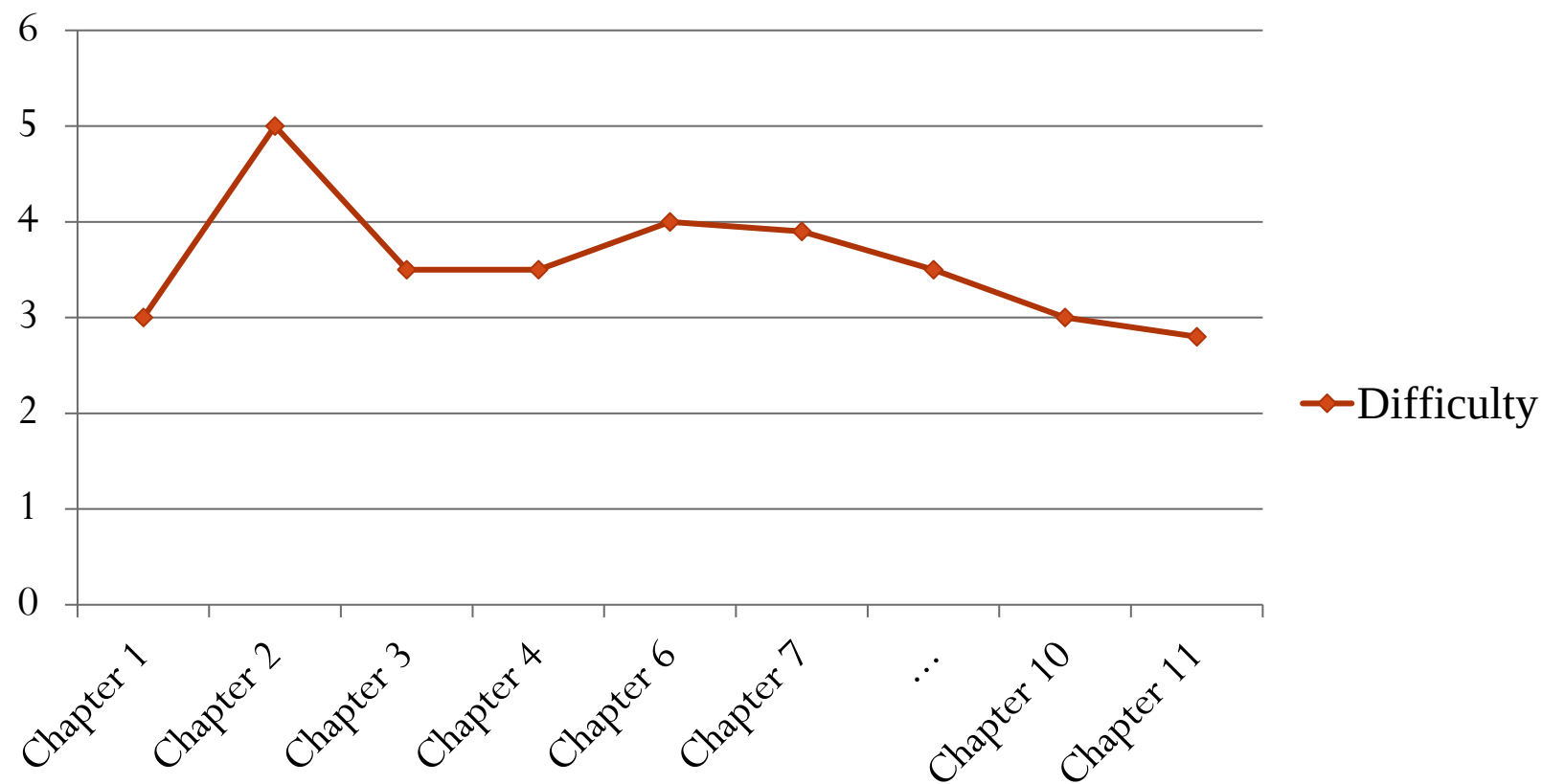
- 运算基础
  - 数制和数制转换: 二进制binary, 十进制decimal, 十六进制hexadecimal
  - 数据存储: 原码（符号）, 反码（减法运算）, 补码（简化硬件）
  - 信息编码方法: BCD, ASCII
  - 二进制算术和逻辑运算规则

## 第二章 X86 CPU 系统架构

**Dr. Prof. Ni Li**

# 学习难度

难度系数

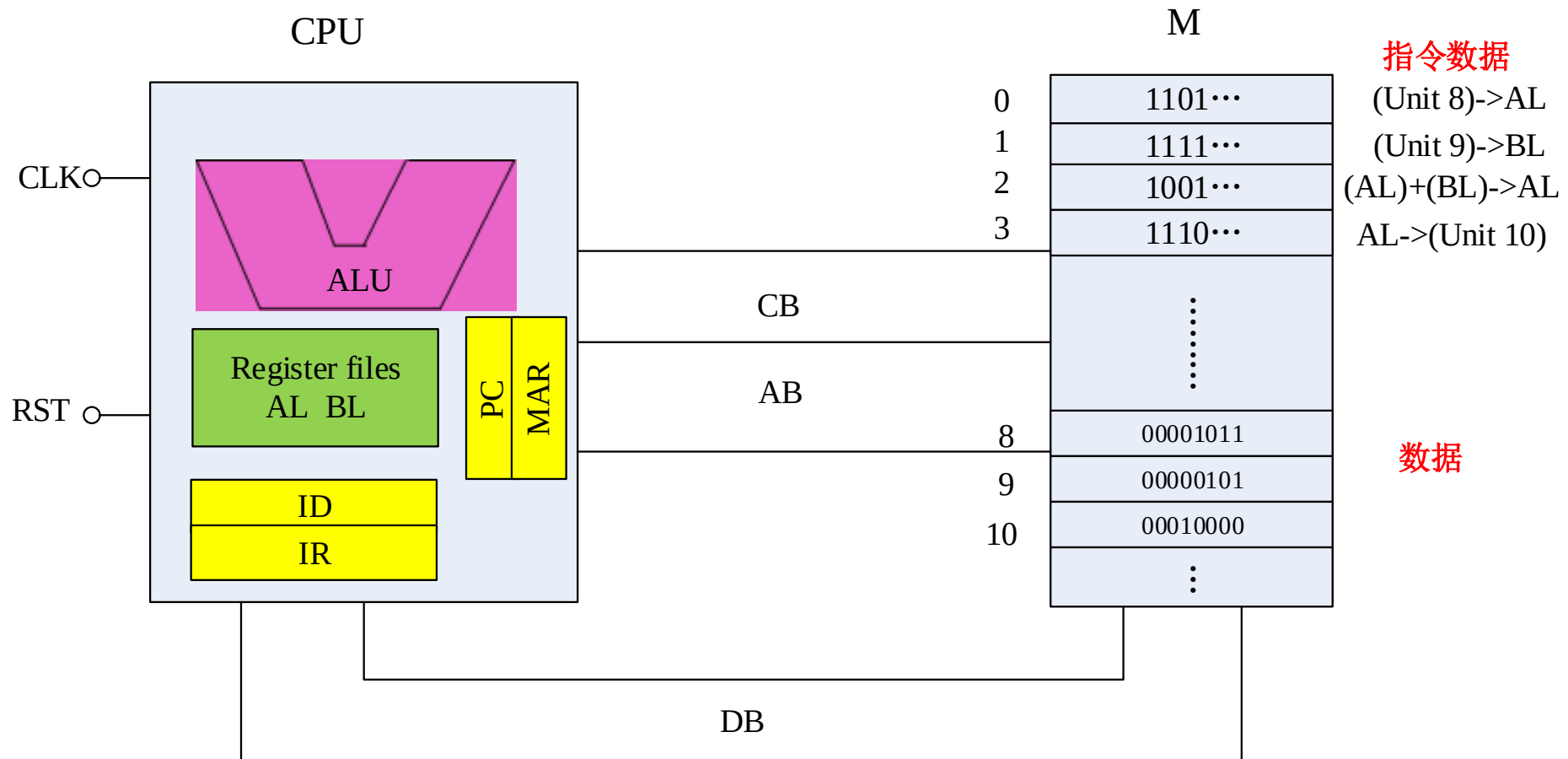


## 2.1 8086系统架构

# 微机工作原理

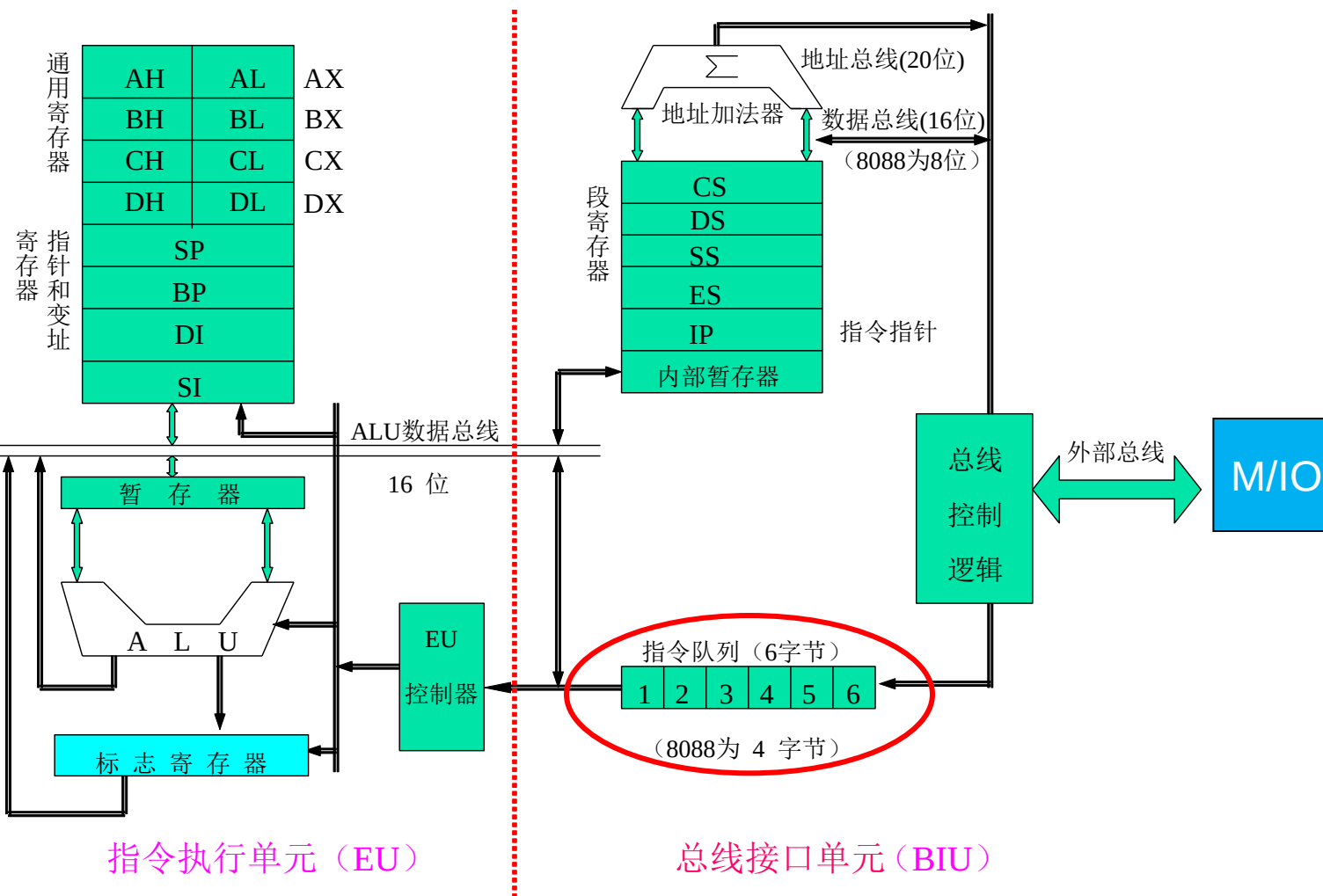
并行处理?

- Examples: 按顺序取指令、译码执行
  - Unit 8->AL; Unit 9->BL; Adding result->Unit 10

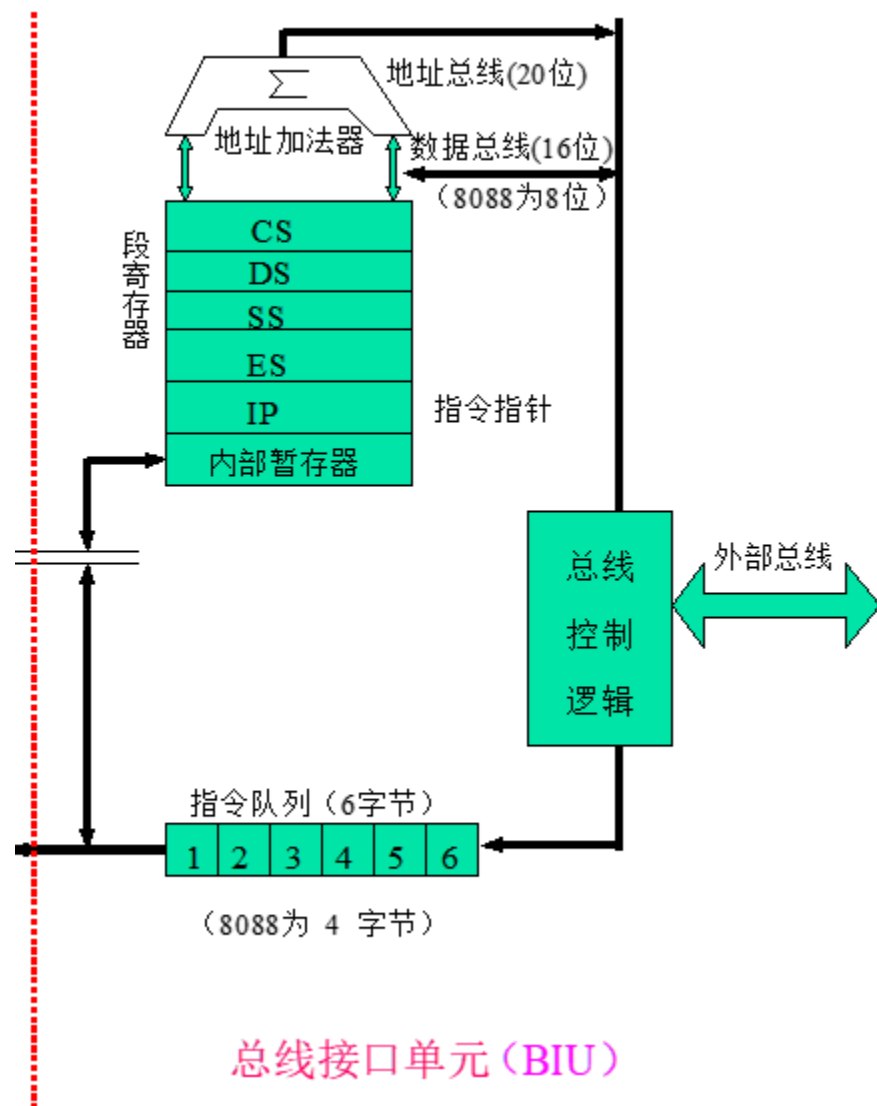


# 8086系统架构

- **EU(Execution Unit)** and **BIU (Bus Interface Unit)**
  - 取指令, 译码执行



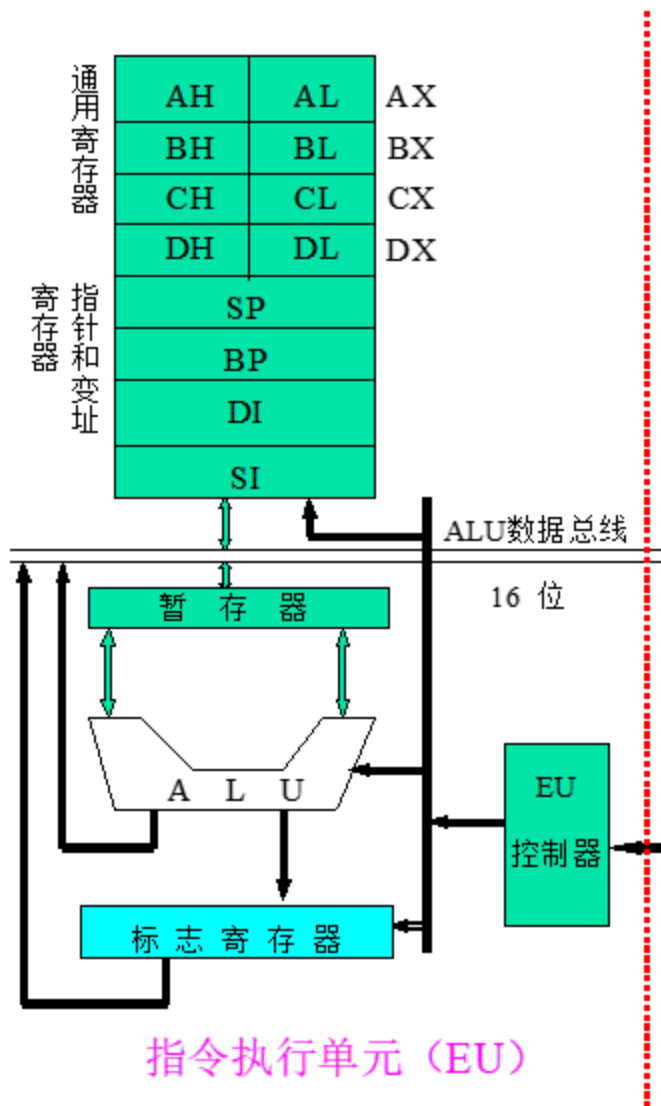
# BIU（总线接口单元）



- 功能
  - 取指令到**指令队列**, 读操作数/写结果数据
  - 提供16-bit 双向DB, 20-bit AB和总线控制信号
- 工作过程
  - CS:IP给出指令地址
  - 指令队列满则BIU空闲, 出现**2个空余字节**则自动取指
- 段寄存器Segment registers
  - CS、DS、ES、SS（提供段地址）
- IP
  - 下一条指令偏移地址
- 地址加法器Address adder
  - 逻辑地址(16 bits) ->物理地址(20 bits)
  - 地址范围?
- **指令队列(6 bytes)**
  - 8086 CPU指令长度范围?



# EU（指令执行单元）



- 译码和执行
- 控制器controller
  - 取指令控制(指令队列)、译码
- ALU
  - 二进制算术逻辑运算
- 寄存器Register files
  - 通用register (AX/BX/CX/DX)
  - 指针Pointers、变址index register (SP/BP/SI/DI)
  - 标志位寄存器FR: 指令执行结果
- 工作过程
  - 指令队列中指令译码
  - ALU运算
  - 提供操作数偏移地址

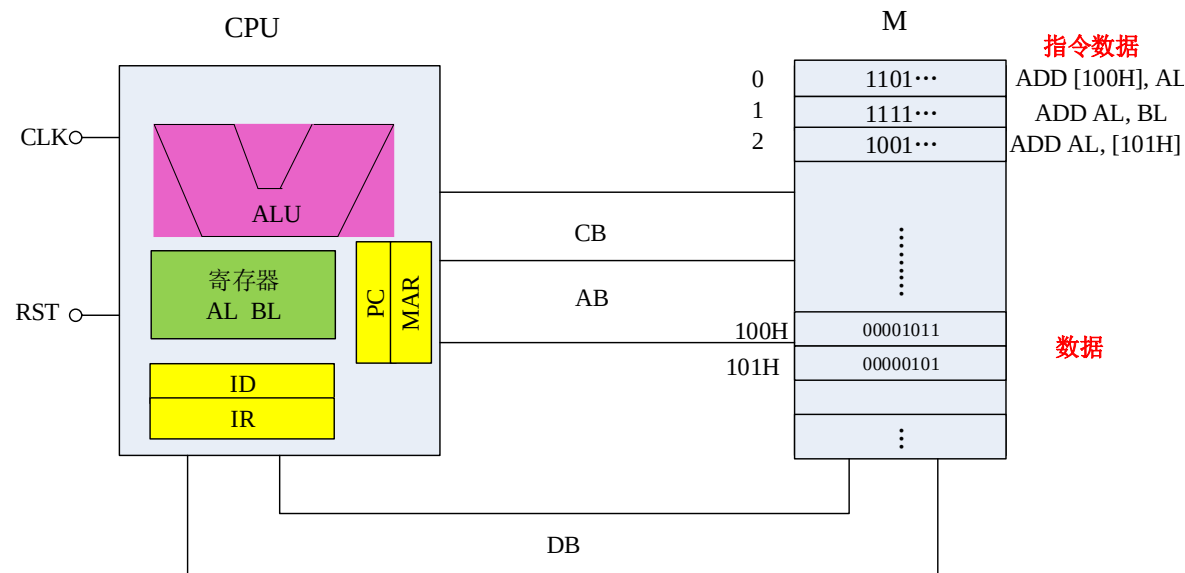
# 并行操作

- BIU + EU → 并行操作
- BIU
  - 取指令, 读写数据(操作数operands, 结果数据result data)
  - 指令队列出现2个空余字节则自动取指
- EU
  - 从指令队列取指令, 译码执行(无须访问Memory or I / O)
  - 节省从Memory or I/O 取指令的时间

# 并行操作

- 指令并行操作

- 1: ADD [100H], AL
- 2: ADD AL, BL
- 3: ADD AL, [101H]

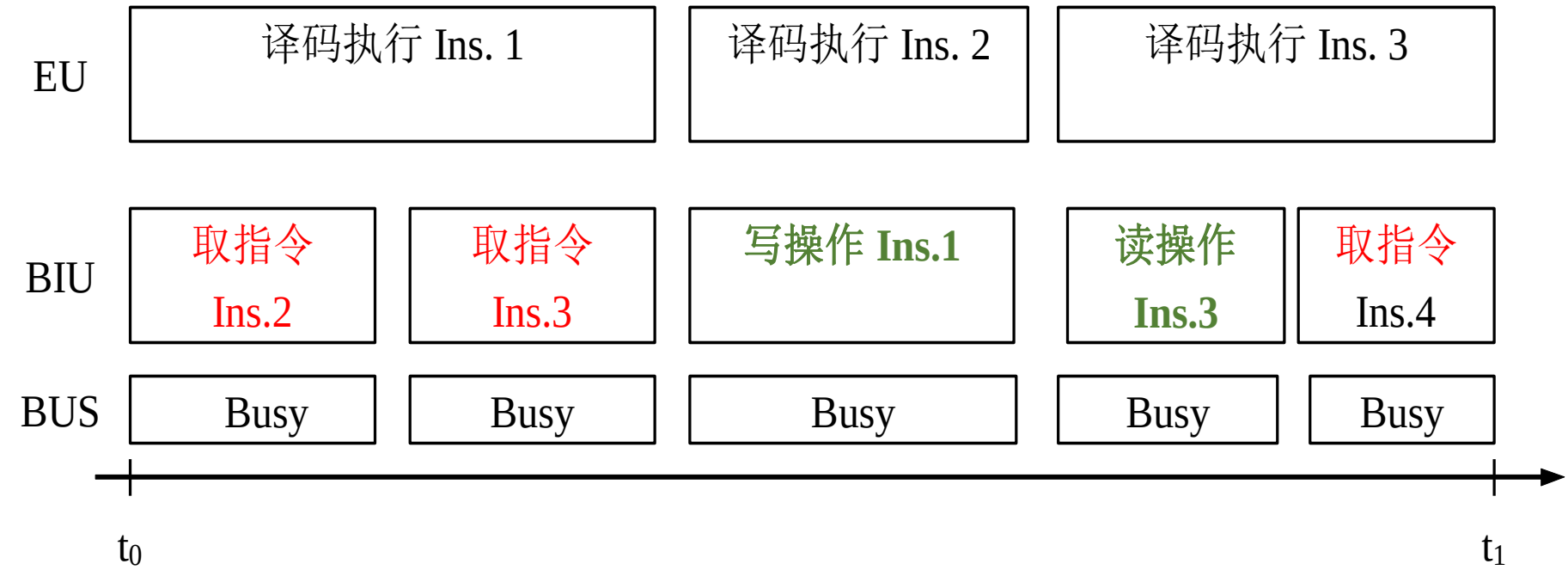


- 指令1:取指令, 译码, 读 [100H], 做加法, 写[100H]
- 指令2:取指令, 译码, 做加法
- 指令3:取指令, 译码, 读 [101H], 做加法

| 指令 | BIU                   | EU     |
|----|-----------------------|--------|
| 1  | 取指令, 读[100H], 写[100H] | 译码, 加法 |
| 2  | 取指令                   | 译码, 加法 |
| 3  | 取指令, 读[101H]          | 译码, 加法 |

# 并行操作

- 指令并行操作



- 1: ADD [100H], AL
- 2: ADD AL, BL
- 3: ADD AL, [101H]

| BIU                   | EU     |
|-----------------------|--------|
| 取指令, 读[100H], 写[100H] | 译码, 加法 |
| 取指令                   | 译码, 加法 |
| 取指令, 读[101H]          | 译码, 加法 |

# 并行操作



# 寄存器

|    |    |   |    |   |              |
|----|----|---|----|---|--------------|
|    | 15 | 8 | 7  | 0 |              |
| AX | AH |   | AL |   | accumulator  |
| BX | BH |   | BL |   | Base address |
| CX | CH |   | CL |   | counter      |
| DX | DH |   | DL |   | data         |

|    |   |                   |
|----|---|-------------------|
| 15 | 0 |                   |
| SP |   | Stack pointer     |
| BP |   | Base pointer      |
| SI |   | Source index      |
| DI |   | Destination index |

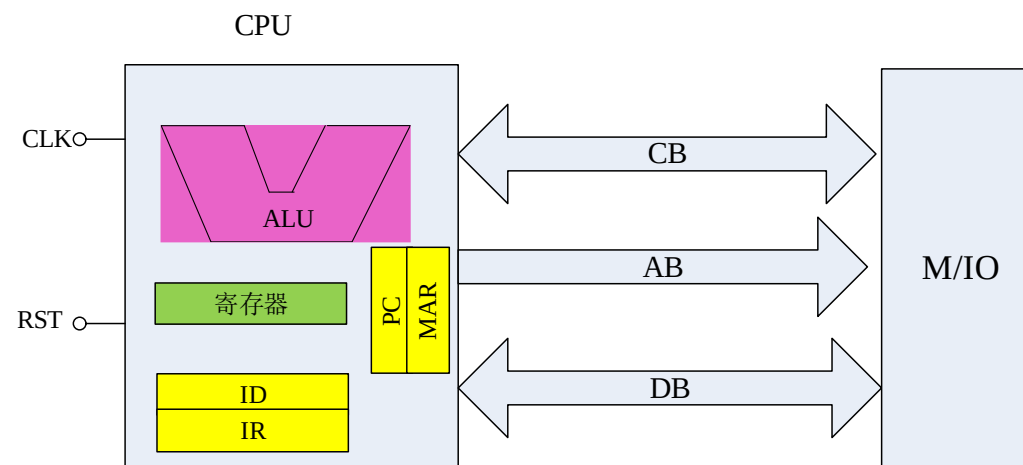
|    |   |               |
|----|---|---------------|
| 15 | 0 |               |
| CS |   | Code segment  |
| DS |   | Data segment  |
| SS |   | Stack segment |
| ES |   | Extra segment |

|    |   |                     |
|----|---|---------------------|
| 15 | 0 |                     |
| IP |   | Instruction pointer |
| FR |   | flag                |

# 通用寄存器—4 16-bit registers

- 在CPU内部: 比内存访问效率高
  - 存储操作数, 操作数地址和中间结果
- AX- accumulator累加器
  - AH, AL
- BX- base address register基址寄存器
  - BH, BL
- CX- counter register计数器寄存器
  - CH, CL
- DX- data register
  - DH, DL



# 通用寄存器的特殊用法

| 寄存器名   | 特殊用途                              |
|--------|-----------------------------------|
| AX, AL | 在输入输出指令中作数据寄存器                    |
| AL     | 在十进制运算指令中作累加器<br>在 XLAT 指令中作累加器   |
| BX     | 在间接寻址中作基址寄存器<br>在 XLAT 指令中作基址寄存器  |
| CX     | 在串操作指令和 LOOP 指令中作计数器              |
| CL     | 在移位/循环移位指令中作移位次数寄存器               |
| DX     | 在间接寻址的输入输出指令中作地址寄存器               |
| SI     | 在字符串运算指令中作源变址寄存器<br>在间接寻址中作变址寄存器  |
| DI     | 在字符串运算指令中作目标变址寄存器<br>在间接寻址中作变址寄存器 |
| BP     | 在间接寻址中作基址指针                       |
| SP     | 在堆栈操作中作堆栈指针                       |



# 内存地址生成

- 存储器分段
  - 存储器分段存储不同数据
- Code segment代码段
  - 存储程序的二进制数据
- Data segment数据段
  - 存储数据, 如操作数、变量
- Extra segment附加段
  - 存储数据
- Stack segment堆栈段
  - 存储临时数据或变量
  - 内存中的特殊空间, 程序执行过程中作为堆栈使用
  - FILO : 先入后出

# 寄存器 Register organization

|    |    |   |    |   |              |
|----|----|---|----|---|--------------|
|    | 15 | 8 | 7  | 0 |              |
| AX | AH |   | AL |   | accumulator  |
| BX | BH |   | BL |   | Base address |
| CX | CH |   | CL |   | counter      |
| DX | DH |   | DL |   | data         |

|  |    |   |                   |
|--|----|---|-------------------|
|  | 15 | 0 |                   |
|  | SP |   | Stack pointer     |
|  | BP |   | Base pointer      |
|  | SI |   | Source index      |
|  | DI |   | Destination index |

|    |   |               |
|----|---|---------------|
| 15 | 0 |               |
| CS |   | Code segment  |
| DS |   | Data segment  |
| SS |   | Stack segment |
| ES |   | Extra segment |

|    |   |                     |
|----|---|---------------------|
| 15 | 0 |                     |
| IP |   | Instruction pointer |
| FR |   | flag                |

# 段寄存器 Segment registers

## — 4 16-bit registers

- Registers: 存储段地址 segment address
  - 段地址 Segment address (段基址 segment base)
  - 逻辑段的起始地址
- CS
  - 代码段 Code segment register
- DS
  - 数据段 Data segment register
- ES
  - 附加段 Extra segment register
- SS
  - 堆栈段 Stack segment register

| M        |           |
|----------|-----------|
|          | .....     |
| 1234H:0H | 11111100B |
| 1234H:1H | 10010011B |
| 1234H:2H | 11101100B |
|          | ...       |
|          | ...       |
|          | ...       |
| 2000H:0H | 00001011B |
| 2000H:1H | 00000101B |
|          | .....     |
|          | ...       |

# 寄存器

|    | 15 | 8 | 7  | 0 |              |
|----|----|---|----|---|--------------|
| AX | AH |   | AL |   | accumulator  |
| BX | BH |   | BL |   | Base address |
| CX | CH |   | CL |   | counter      |
| DX | DH |   | DL |   | data         |

| 15 | 0 |               |
|----|---|---------------|
| CS |   | Code segment  |
| DS |   | Data segment  |
| SS |   | Stack segment |
| ES |   | Extra segment |

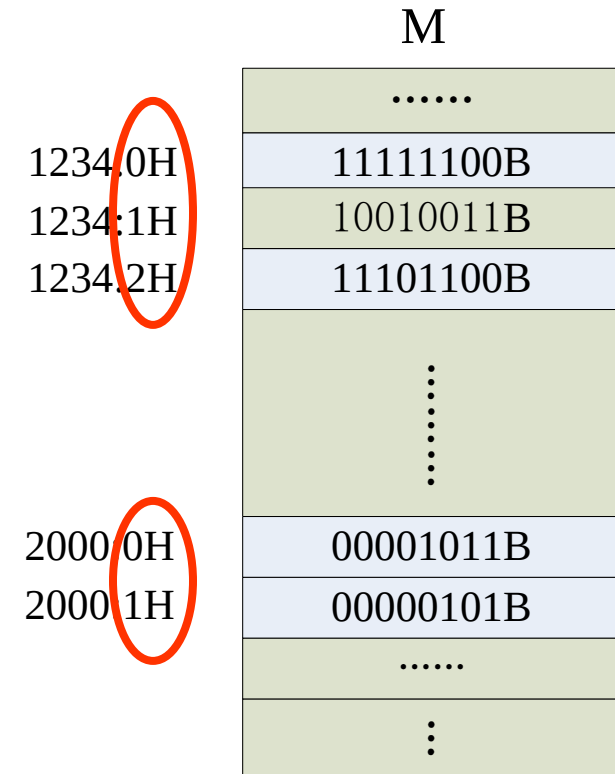
| 15 | 0 |                   |
|----|---|-------------------|
| SP |   | Stack pointer     |
| BP |   | Base pointer      |
| SI |   | Source index      |
| DI |   | Destination index |

| 15 | 0 |                     |
|----|---|---------------------|
| IP |   | Instruction pointer |
| FR |   | flag                |

# 指令Pointers和变址index registers

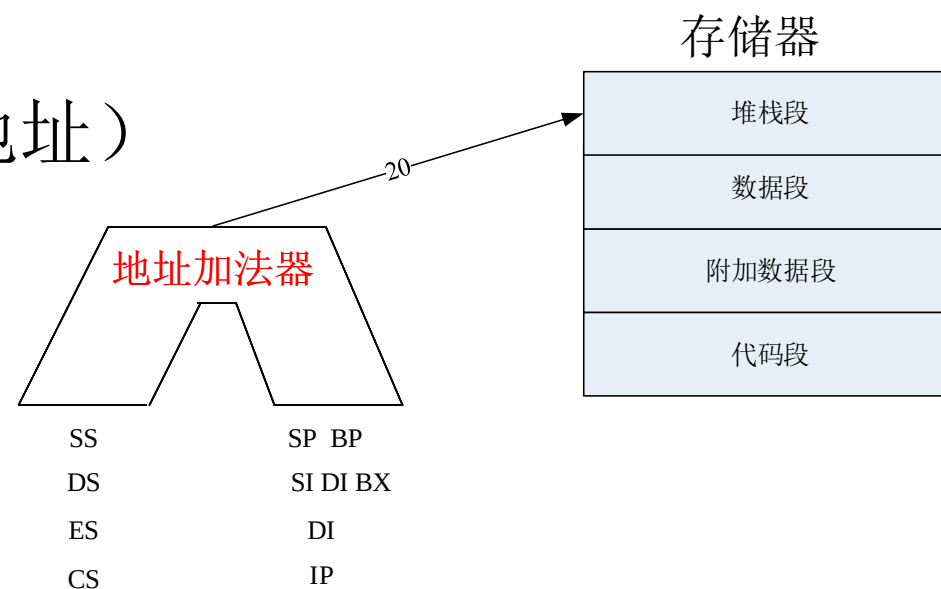
## — 4 16-bit registers

- 存储偏移地址Offset address
  - 数据段、堆栈段、附加段
- BP
  - Base pointer register基址指针寄存器
- SP
  - Stack pointer register堆栈指针寄存器
- SI
  - Source index register源变址寄存器
- DI
  - Destination index register目的变址寄存器



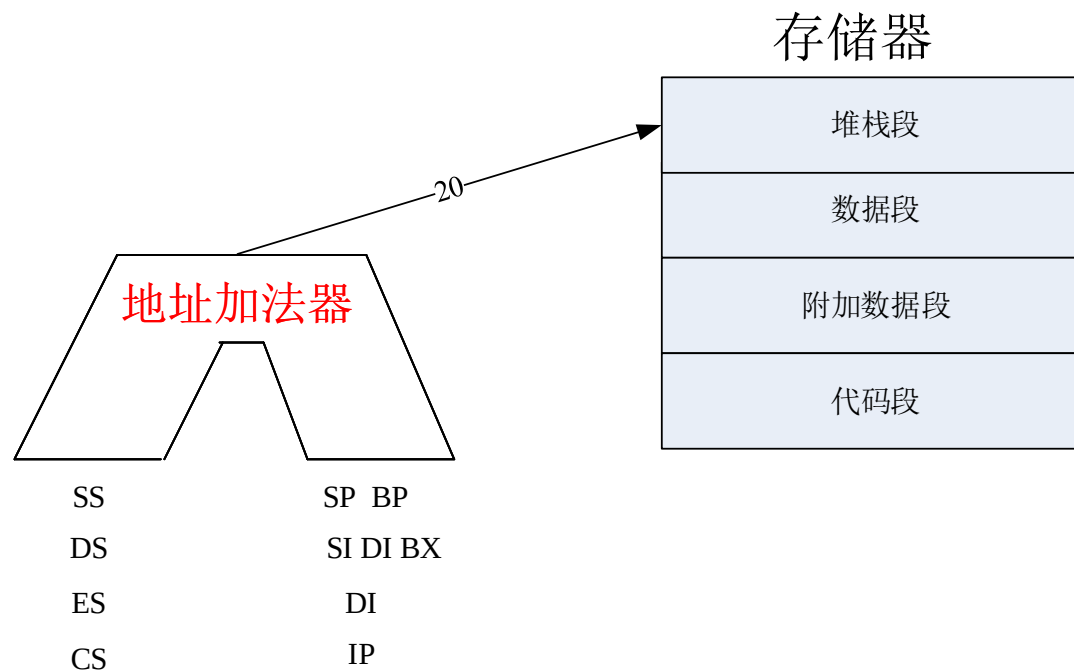
# 内存地址生成

- 内存分段使用Segmented memory
  - 16-bit 段地址 与 16-bit 偏移地址**共同形成**20-bit 物理地址
- 缺省搭配（汇编编程时表示内存地址）
  - Code segment: CS and IP（取指令用）
  - Data segment: DS and SI, DI, BX
  - Extra segment: ES and DI
  - Stack segment: SS and SP, BP
- 20-bit物理地址生成**
  - 存储器地址 = 段基址 (SR)  $\times$  10H + 偏移地址 (pointer/index R)



# 内存地址生成

- **Examples: CS=1234H IP=100H**
  - 20位地址总线上信号A<sub>19</sub>A<sub>18</sub>A<sub>17</sub>.... A<sub>1</sub>A<sub>0</sub> ?



# 寄存器 Register organization

|    |    |   |    |   |              |
|----|----|---|----|---|--------------|
|    | 15 | 8 | 7  | 0 |              |
| AX | AH |   | AL |   | accumulator  |
| BX | BH |   | BL |   | Base address |
| CX | CH |   | CL |   | counter      |
| DX | DH |   | DL |   | data         |

|    |   |               |
|----|---|---------------|
| 15 | 0 |               |
| CS |   | Code segment  |
| DS |   | Data segment  |
| SS |   | Stack segment |
| ES |   | Extra segment |

|    |   |                   |
|----|---|-------------------|
| 15 | 0 |                   |
| SP |   | Stack pointer     |
| BP |   | Base pointer      |
| SI |   | Source index      |
| DI |   | Destination index |

|    |   |                     |
|----|---|---------------------|
| 15 | 0 |                     |
| IP |   | Instruction pointer |
| FR |   | flag                |



# 指令指针寄存器

Instruction pointer register----16-bit

- IP
  - 存储代码段中下一条指令的偏移地址
- 由BIU自动修改
- 程序不能直接访问IP
  - 程序不可以读，不可以直接修改
  - 有的指令可以改变IP: JMP

# 寄存器 Register organization

|    |    |   |    |   |              |
|----|----|---|----|---|--------------|
|    | 15 | 8 | 7  | 0 |              |
| AX | AH |   | AL |   | accumulator  |
| BX | BH |   | BL |   | Base address |
| CX | CH |   | CL |   | counter      |
| DX | DH |   | DL |   | data         |

|    |   |               |
|----|---|---------------|
| 15 | 0 |               |
| CS |   | Code segment  |
| DS |   | Data segment  |
| SS |   | Stack segment |
| ES |   | Extra segment |

|    |   |                   |
|----|---|-------------------|
| 15 | 0 |                   |
| SP |   | Stack pointer     |
| BP |   | Base pointer      |
| SI |   | Source index      |
| DI |   | Destination index |

|    |   |                     |
|----|---|---------------------|
| 15 | 0 |                     |
| IP |   | Instruction pointer |
| FR |   | flag                |

# 标志位寄存器Flag register---16-bit

- FR
  - 9 个有效位
- 6 个状态标志
  - CF, PF, AF, ZF, SF, OF
- 3 个控制标志
  - 控制EU的特定操作
  - TF, IF, DF

# 标志位寄存器FR

|    |  |  |  |  |    |    |    |    |    |    |  |    |  |    |  |    |
|----|--|--|--|--|----|----|----|----|----|----|--|----|--|----|--|----|
| 15 |  |  |  |  | 11 | 10 | 9  | 8  | 7  | 6  |  | 4  |  | 2  |  | 0  |
|    |  |  |  |  | OF | DF | IF | TF | SF | ZF |  | AF |  | PF |  | CF |

| 标志名 | 作用                | 标志为 1 | 标志为 0 |
|-----|-------------------|-------|-------|
| CF  | 运算结果 进位标志         | 进/借位  |       |
| PF  | 运算结果低 8 位 奇偶标志    | 偶个 1  | 奇个 1  |
| AF  | 辅助进位，低 4 位向 4 位进位 | 进/借位  |       |
| ZF  | 运算结果 全零标志         | 全零    |       |
| SF  | 运算结果 符号（正负）       | 负     | 正     |
| OF  | 运算结果 溢出标志         | 溢出    |       |
| TF  | CPU 状态，单步模式       | 允许单步  | 不允许单步 |
| IF  | CPU 状态，中断屏蔽       | 允许中断  | 不允许中断 |
| DF  | CPU 状态，方向标志       | 地址减   | 地址加   |

# 标志位寄存器FR

- **Examples**

$$\begin{array}{r} 10000001 \\ + 11111101 \\ \hline 1\ 01111110 \end{array}$$

- 有符号数
- 无符号数:  $129+253=382$   
SF & OF: 无意义  
CF: 作为结果一部分



-127+(-3)  
CF=1 PF=1 AF=0  
ZF=0 SF=0 OF=1

## 2.2 8086管脚

# 8086管脚

|      |    |    |                                         |
|------|----|----|-----------------------------------------|
| GND  | 1  | 40 | VCC(5v)                                 |
| AD14 | 2  | 39 | AD15                                    |
| AD13 | 3  | 38 | A16/S3                                  |
| AD12 | 4  | 37 | A17/S4                                  |
| AD11 | 5  | 36 | A18/S5                                  |
| AD10 | 6  | 35 | A19/S6                                  |
| AD9  | 7  | 34 | BHE/S7                                  |
| AD8  | 8  | 33 | <b>MN/MX</b>                            |
| AD7  | 9  | 32 | $\overline{\text{RD}}$                  |
| AD6  | 10 | 31 | HOLD ( <u>RQ/GT0</u> )                  |
| AD5  | 11 | 30 | HLDA ( <u>RQ/GT1</u> )                  |
| AD4  | 12 | 29 | $\overline{\text{WR}}$ ( <u>LOCK</u> )  |
| AD3  | 13 | 28 | $\overline{\text{M/IO}}$ ( <u>S2</u> )  |
| AD2  | 14 | 27 | $\overline{\text{DT/R}}$ ( <u>S1</u> )  |
| AD1  | 15 | 26 | DEN ( <u>S0</u> )                       |
| AD0  | 16 | 25 | ALE ( <u>QS0</u> )                      |
| NMI  | 17 | 24 | $\overline{\text{INTA}}$ ( <u>QS1</u> ) |
| INTR | 18 | 23 | TEST                                    |
| CLK  | 19 | 22 | READY                                   |
| GND  | 20 | 21 | RESET                                   |

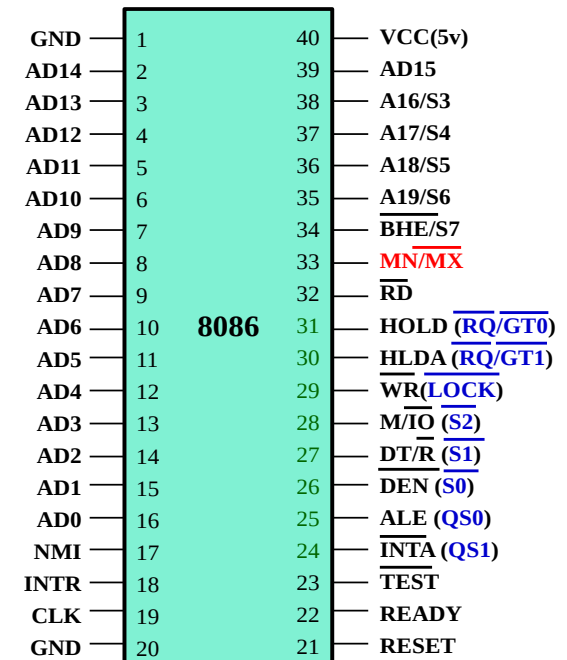
最小模式 (最大模式)



- 8086 CPU
  - 20-bit 地址线
  - 16-bit 数据线
  - 读写控制R/W Control signals
  - 供电Power supply
  - GND
- 封装Package type
  - 40-pin双列直插式
  - Dual In-Line Package (DIP)
- 分时复用

# 8086管脚

- 1.  $\overline{MN}/\overline{MX}$ 
  - MN/MX mode control最小/最大模式控制
    - Minimum mode: 单处理器系统, 提供所有信号 (AB, CB, DB)
    - Maximum mode: 多处理器系统, 连接到总线控制器
    - Input
- 2. AD15~AD0: 地址数据复用管脚
  - 分时复用
  - 三态Three-state (高、低、高阻态)
  - 输出地址信号, 传输数据信号



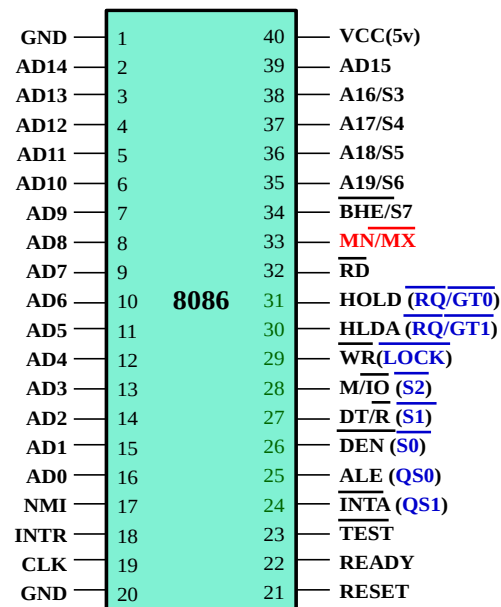
minimum mode (maximum mode)



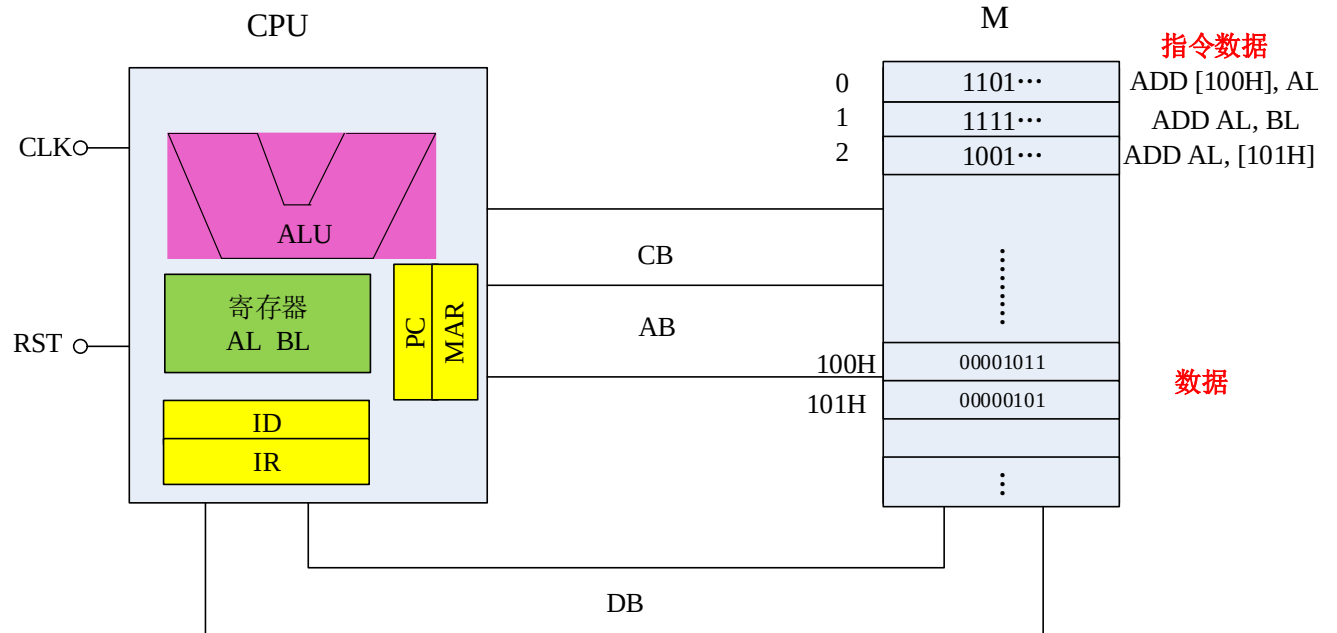
# 8086管脚

- 2. AD15~AD0: 地址数据复用管脚
  - 通过总线读取数据需要4个T
  - 先输出地址信号(T1), 再传输数据信号(T2-T4)
  - Examples**
    - DS=100H, SI=100H, AL=55H, 执行指令

**MOV [SI], AL      AD15-0 ?**



minimum mode (maximum mode)



# 8086管脚

- 3. ALE: Address Latch Enable

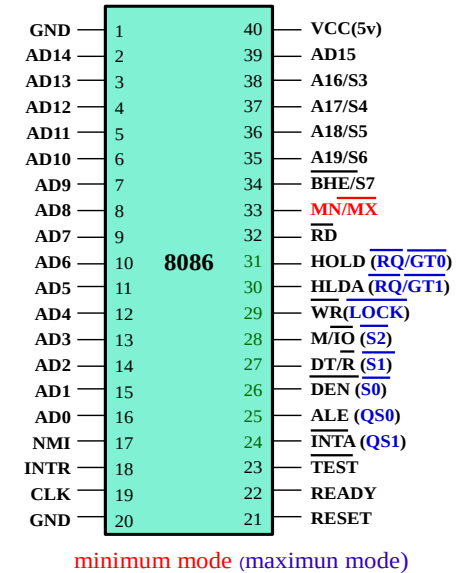
- Output
- Active-high
- 因为有管脚分时复用->需要信号控制**锁存器**
- 地址锁存器**74LS373**的片选信号

- 4.  $\overline{DT/R}$  : Data Transmit/Receive

- Output
- 控制**数据收发器**的数据传送方向(提高数据带载能力) **74LS245**
- 读/写: 站在CPU角度

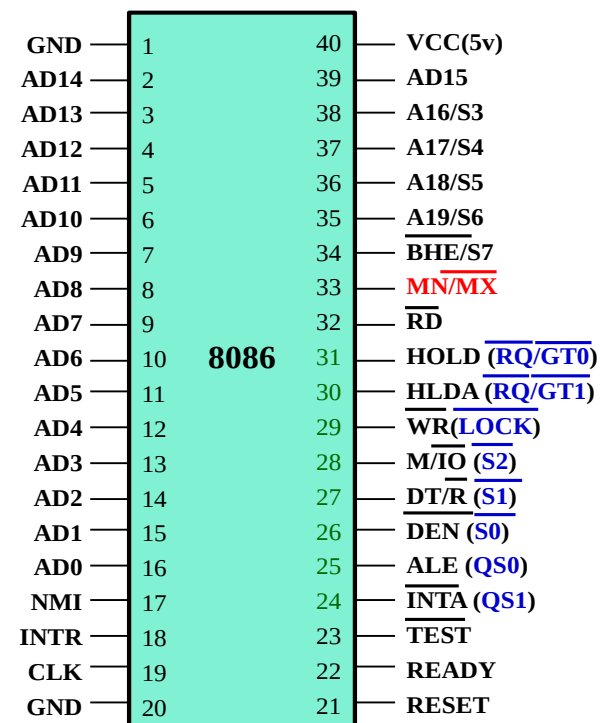
- 5.  $\overline{DEN}$  : Data Enable

- 地址信号消失后, 需要传输数据信号时再给出低有效
- **数据收发器** 片选信号 (**74LS245**)
- 双向数据传输Bi-directional transfer



# 8086管脚

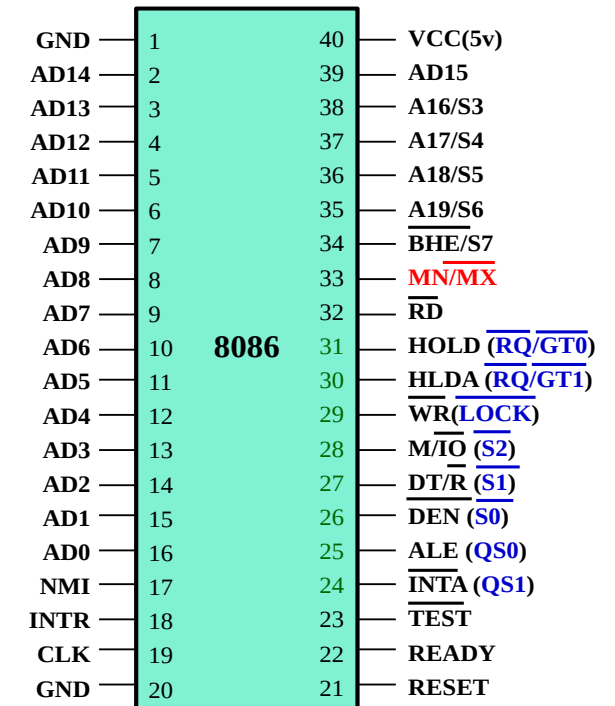
- 6. A19-16/S6-3
  - 地址状态复用信号
  - 与A15-0一起提供20-bit地址信号
  - 访问I/O port (I/O: 16-bit 地址)时未用到
  - 先输出地址, 后输出状态
- S6 S5 S4 S3: 参考教材



minimum mode (maximun mode)

# 8086管脚

- 7. BHE/S7: Bus High Enable/Status
  - 奇地址选通信号（高8位总线有效）
    - 低电平时选通M 或 I/O 端口的奇地址
    - S7: no definition

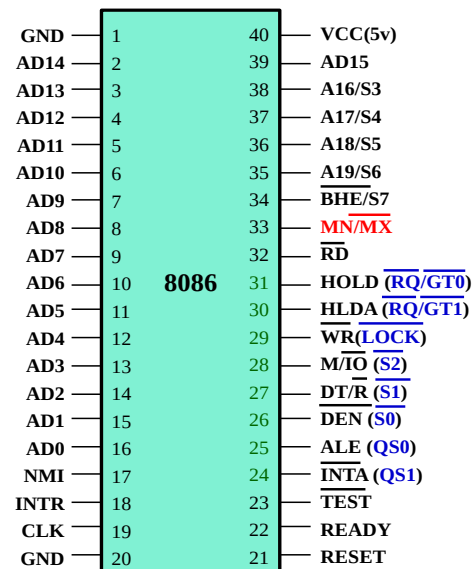


minimum mode (maximum mode)

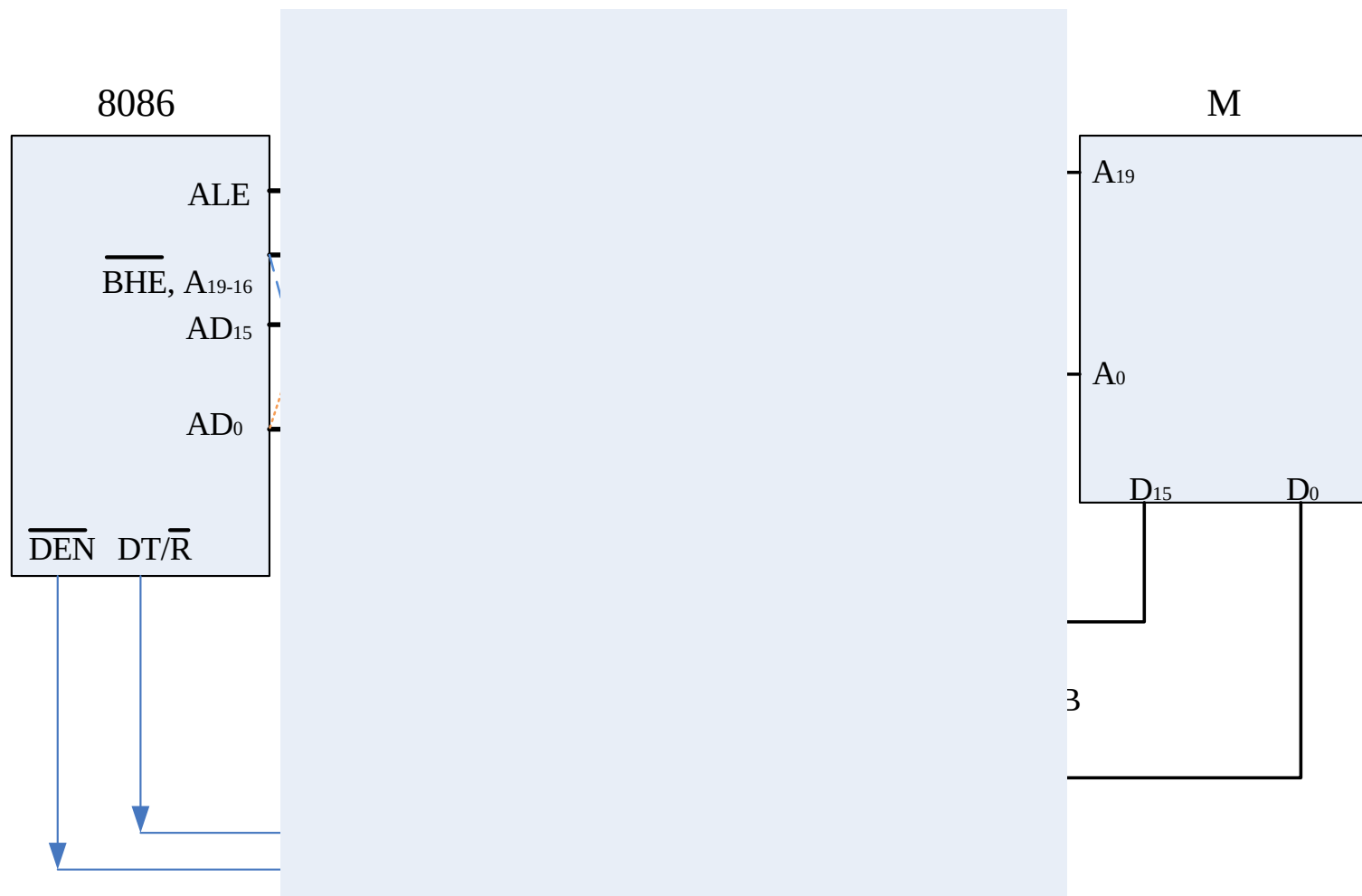
# 8086管脚

- 理解相关管脚连接关系

- AD AS
- BHE/S7
- ALE
- DEN(Data)
- DT/R

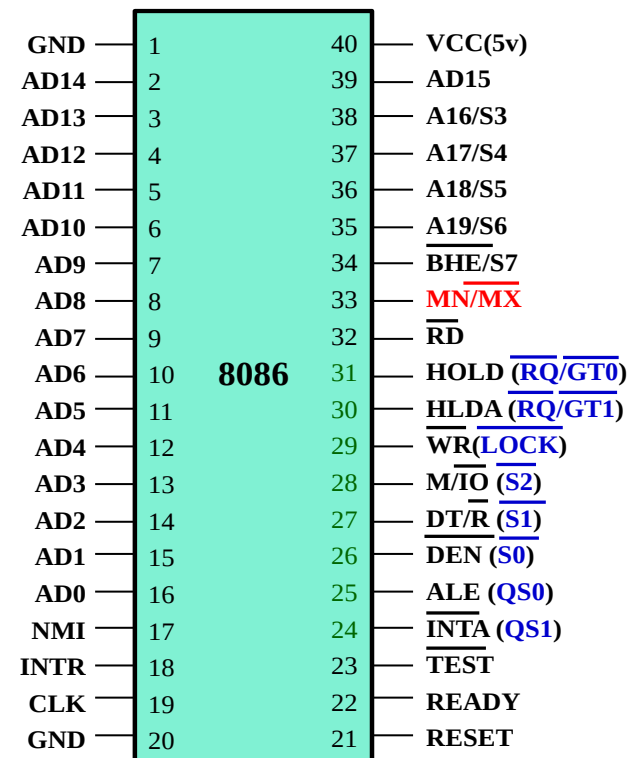


minimum mode (maximum mode)



# 8086管脚

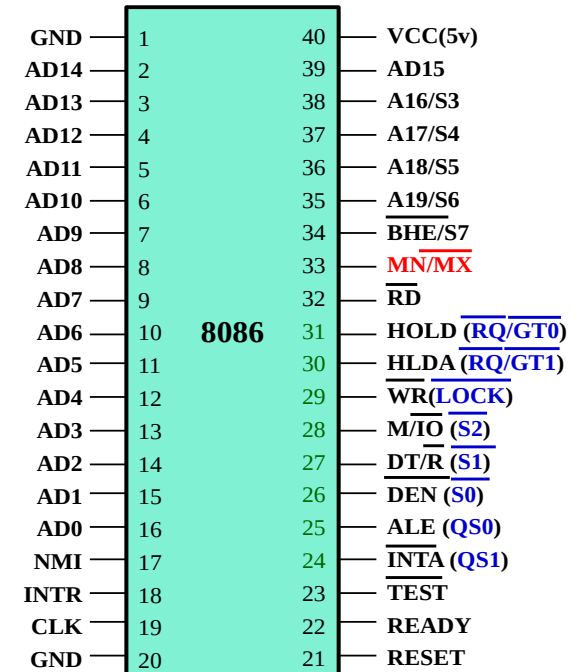
- 8.  $\overline{RD}$ 
  - Output
  - 低电平有效Active-low
- 9.  $\overline{WR}$ 
  - Output
  - 低电平有效Active-low
- 10.  $M/\overline{IO}$ 
  - Output
  - M 和 I/O 操作控制信号



minimum mode (maximum mode)

# 8086管脚

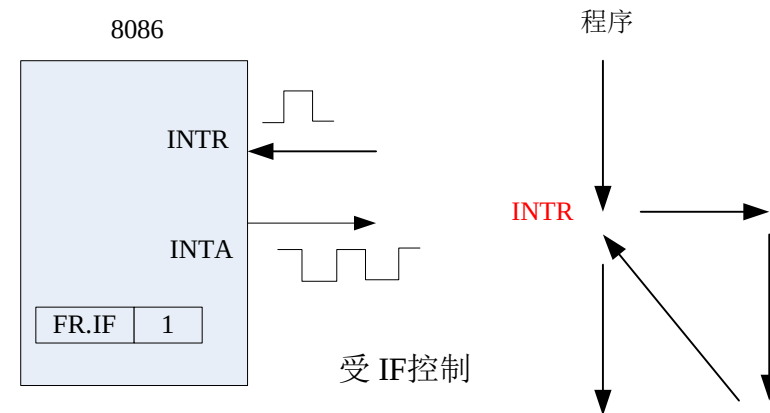
- 11. READY
  - Input
  - Active-high
  - 从M/IO应答信号: 存储器或外设准备好进行数据传输, 检查READY
    - 总线周期Bus cycle: 通过数据总线进行读写所需要的时间
    - 一般包括4个时钟周期(T<sub>1</sub>-T<sub>4</sub>)
    - T<sub>3</sub>
    - 插入 T<sub>w</sub>



minimum mode (maximum mode)

# 8086管脚

- 12. INTR: Interrupt Request
  - 可屏蔽中断请求信号
  - Input
  - 高电平有效Active-high
  - CPU检测INTR
    - 指令周期的最后一个时钟周期检测
    - 若 INTR 有效且IF=1, CPU 在完成当前指令后响应中断
- 13. INTA: Interrupt Acknowledge
  - Output
  - 低电平有效Active-low
  - 应答过程



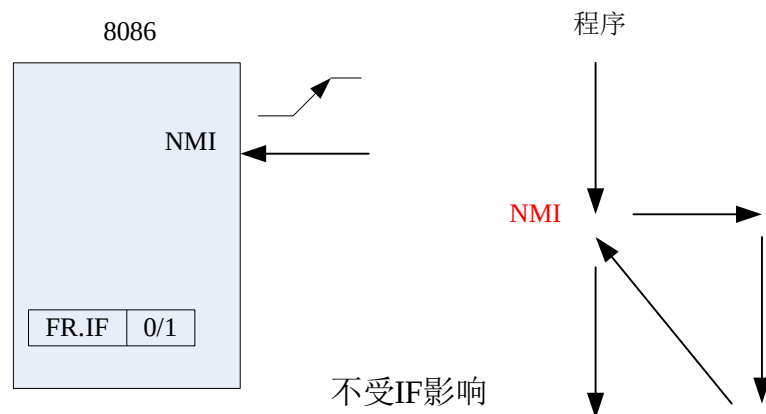
|      |    |    |                                    |
|------|----|----|------------------------------------|
| GND  | 1  | 40 | VCC(5v)                            |
| AD14 | 2  | 39 | AD15                               |
| AD13 | 3  | 38 | A16/S3                             |
| AD12 | 4  | 37 | A17/S4                             |
| AD11 | 5  | 36 | A18/S5                             |
| AD10 | 6  | 35 | A19/S6                             |
| AD9  | 7  | 34 | BHE/S7                             |
| AD8  | 8  | 33 | <del>MN/MX</del>                   |
| AD7  | 9  | 32 | <del>RD</del>                      |
| AD6  | 10 | 31 | HOLD ( <del>RQ/GT0</del> )         |
| AD5  | 11 | 30 | HLDA ( <del>RQ/GT1</del> )         |
| AD4  | 12 | 29 | <del>WR</del> (LOCK)               |
| AD3  | 13 | 28 | M/ <del>IO</del> ( <del>S2</del> ) |
| AD2  | 14 | 27 | <del>DT/R</del> ( <del>S1</del> )  |
| AD1  | 15 | 26 | <del>DEN</del> ( <del>S0</del> )   |
| AD0  | 16 | 25 | ALE ( <del>QS0</del> )             |
| NMI  | 17 | 24 | <del>INTA</del> ( <del>QS1</del> ) |
| INTR | 18 | 23 | TEST                               |
| CLK  | 19 | 22 | READY                              |
| GND  | 20 | 21 | RESET                              |

minimum mode (maximum mode)



# 8086管脚

- 14. NMI: 不可屏蔽中断请求
  - Input
  - 上升沿触发
  - IF 对NMI无影响，不能通过软件屏蔽：通常用于硬件故障处理
  - 响应过程



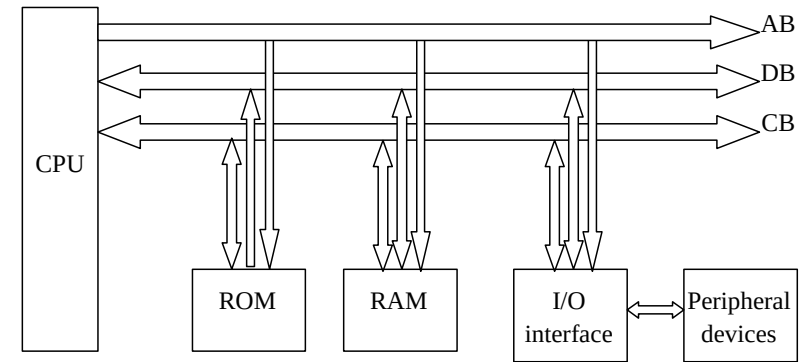
|      |    |    |                                         |
|------|----|----|-----------------------------------------|
| GND  | 1  | 40 | VCC(5v)                                 |
| AD14 | 2  | 39 | AD15                                    |
| AD13 | 3  | 38 | A16/S3                                  |
| AD12 | 4  | 37 | A17/S4                                  |
| AD11 | 5  | 36 | A18/S5                                  |
| AD10 | 6  | 35 | A19/S6                                  |
| AD9  | 7  | 34 | BHE/S7                                  |
| AD8  | 8  | 33 | <b>MN/MX</b>                            |
| AD7  | 9  | 32 | $\overline{\text{RD}}$                  |
| AD6  | 10 | 31 | HOLD ( <u>RQ/GT0</u> )                  |
| AD5  | 11 | 30 | HLDA ( <u>RQ/GT1</u> )                  |
| AD4  | 12 | 29 | $\overline{\text{WR}}$ ( <u>LOCK</u> )  |
| AD3  | 13 | 28 | $\overline{\text{M/IO}}$ ( <u>S2</u> )  |
| AD2  | 14 | 27 | $\overline{\text{DT/R}}$ ( <u>S1</u> )  |
| AD1  | 15 | 26 | DEN ( <u>S0</u> )                       |
| AD0  | 16 | 25 | ALE ( <u>QS0</u> )                      |
| NMI  | 17 | 24 | $\overline{\text{INTA}}$ ( <u>QS1</u> ) |
| INTR | 18 | 23 | TEST                                    |
| CLK  | 19 | 22 | READY                                   |
| GND  | 20 | 21 | RESET                                   |

minimum mode (maximum mode)

# 8086管脚

- 15. HOLD: Hold Request

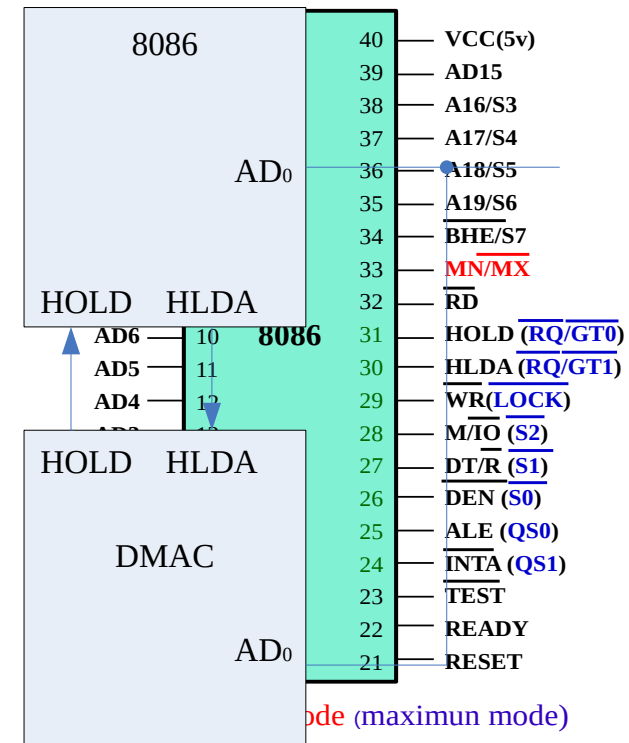
- 总线保持请求Input, Active-high
- 表示其他器件在请求使用 AB, DB 和 CB, 直接与存储器传输数据
- DMA (Direct Memory Access) Controller: 批量数据传输



- 16. HLDA: Hold Acknowledge

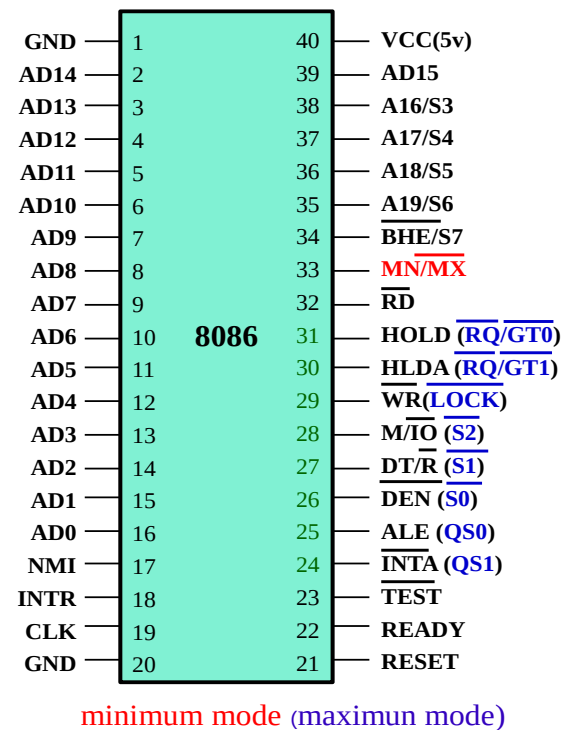
- 总线保持响应Output, Active-high
- Acknowledge process**
  - 发 HLDA 信号 (high)
  - CPU 放弃总线控制权
  - AB, DB 和 CB 由其他部件控制
  - HLDA 变低, CPU 重新控制总线

- 连接图(与 DMAC)**

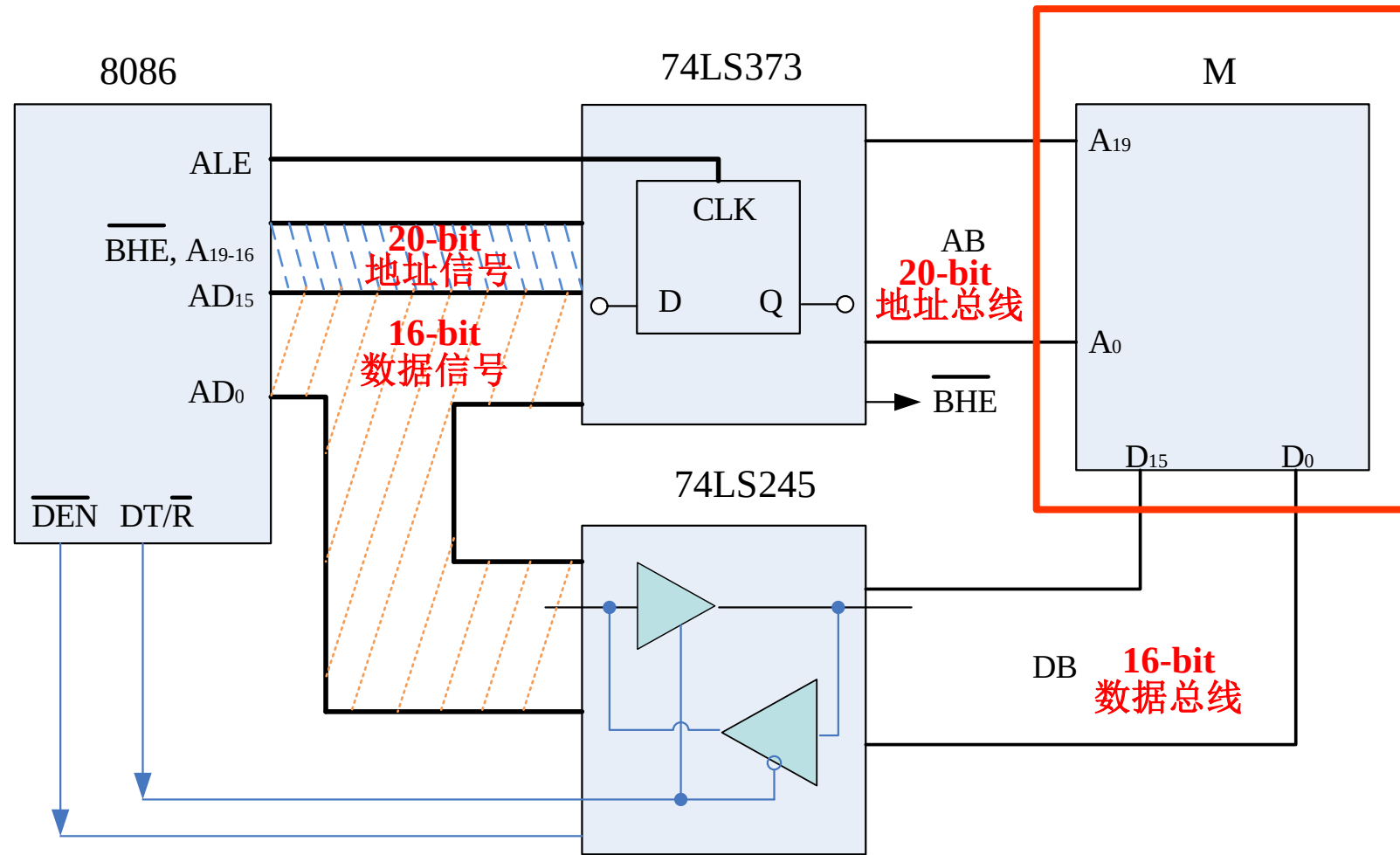


# 8086管脚

- 17. RST
  - Reset, active-high
  - 复位操作
    - DS、ES、SS、FR、IP、CS (FFFFH)
    - 重置时指令地址?
- 18. CLK: Clock
  - Input: 来自8284时钟发生器
- 19. TEST
  - Input, active-low
  - WAIT指令: 用于CPU与外部设备同步
    - CPU 检测 TEST 管脚: wait指令执行过程中每5个 clock cycles检测
    - 若信号为高则空转等待; 否则继续执行后续指令
- 20. Vcc(+5V),  $\overline{\text{GND}}$



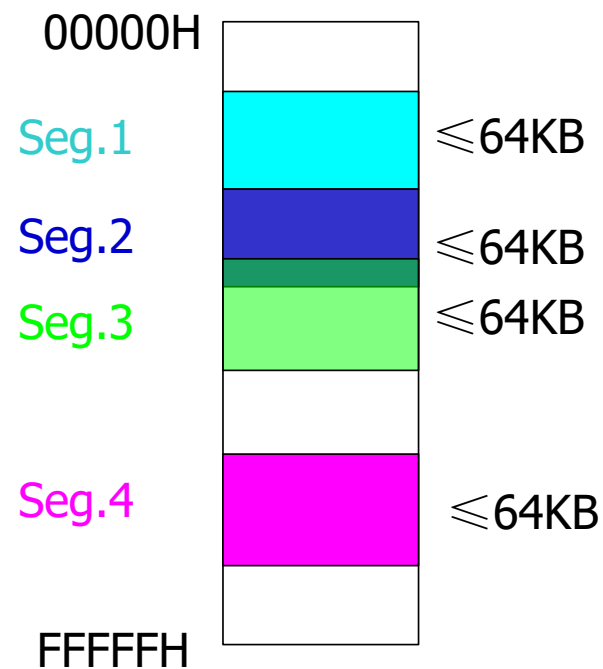
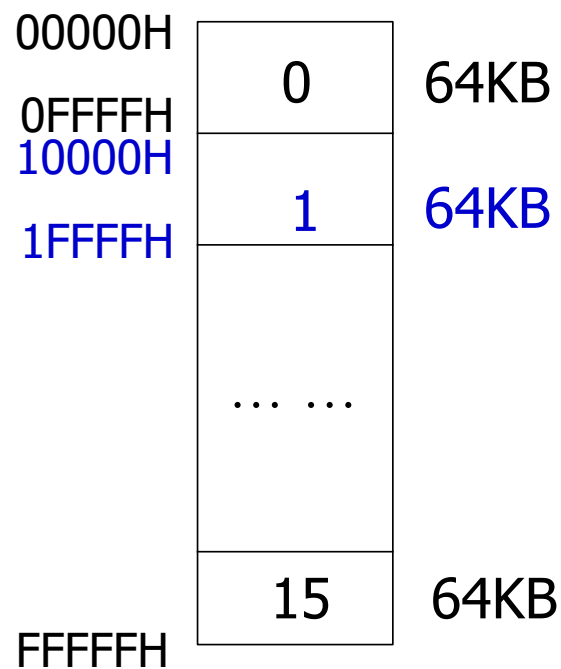
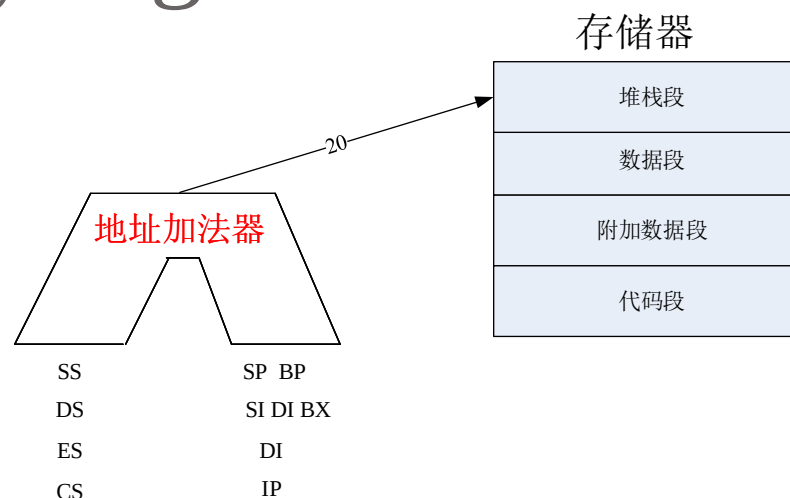
# 8086管脚(最小模式)



## 2.3 8086 存储器结构

# 存储器分段Memory segmentation

- 存储器分段
  - 连续、分开或重叠
- 段最大容量?
  - 偏移地址范围?
  - 16 bits



# 存储器分段Memory segmentation

- 存储单元地址
  - Physical address物理地址
  - Logical address逻辑地址
- 物理地址: 实际地址 (地址总线信号)
  - 20-bit AB: e.g. 12340H
  - 8086 地址空间: 1MB **(Byte)** 1B -> 8 bits
- 逻辑地址
  - 段基址(段地址): 偏移地址
  - 16-bit段基址: 16-bit偏移地址
  - $\text{Physical address} = \text{Segment base} \times 16 (10H) + \text{segment offset}$
  - CPU内表示地址的寄存器、数据总线宽度都是16位, 汇编语言编程时使用逻辑地址表示存储器地址

# 存储器分段Memory segmentation

- 存储单元地址

- **Examples**

- CS=1234H, IP=0 ~ FFFFH, 物理地址范围?

- **Default assignment**

- 1个物理地址可以由多个逻辑地址表示

$$\begin{array}{r} 1234\text{H} \\ \times \quad 10\text{H} \\ \hline 0000 \\ 1234 \\ \hline 12340\text{H} \end{array}$$





# 8086 存储器架构

- **8086 存储器结构**

- 奇 / 偶 存储体 (各512KB, 8-bit 数据线)

- 偶 存储体 ----low 8-bit数据线 存储器宽度—8位

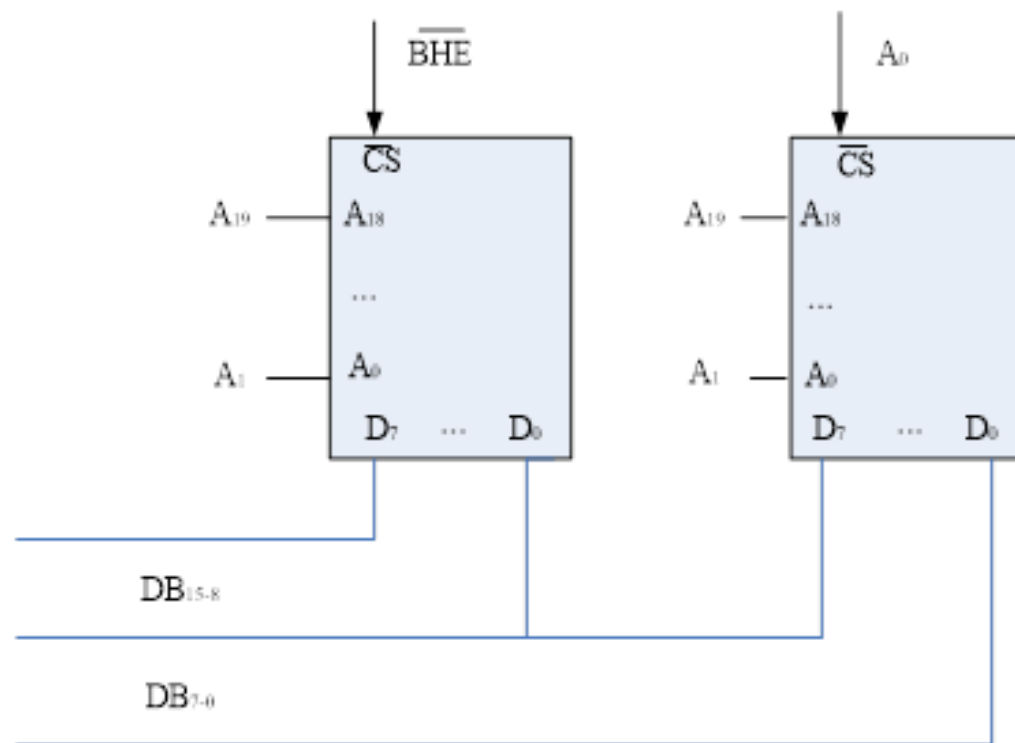
- 奇 存储体 ----high 8-bit数据线

- BHE/A0

- 片选信号

- AB

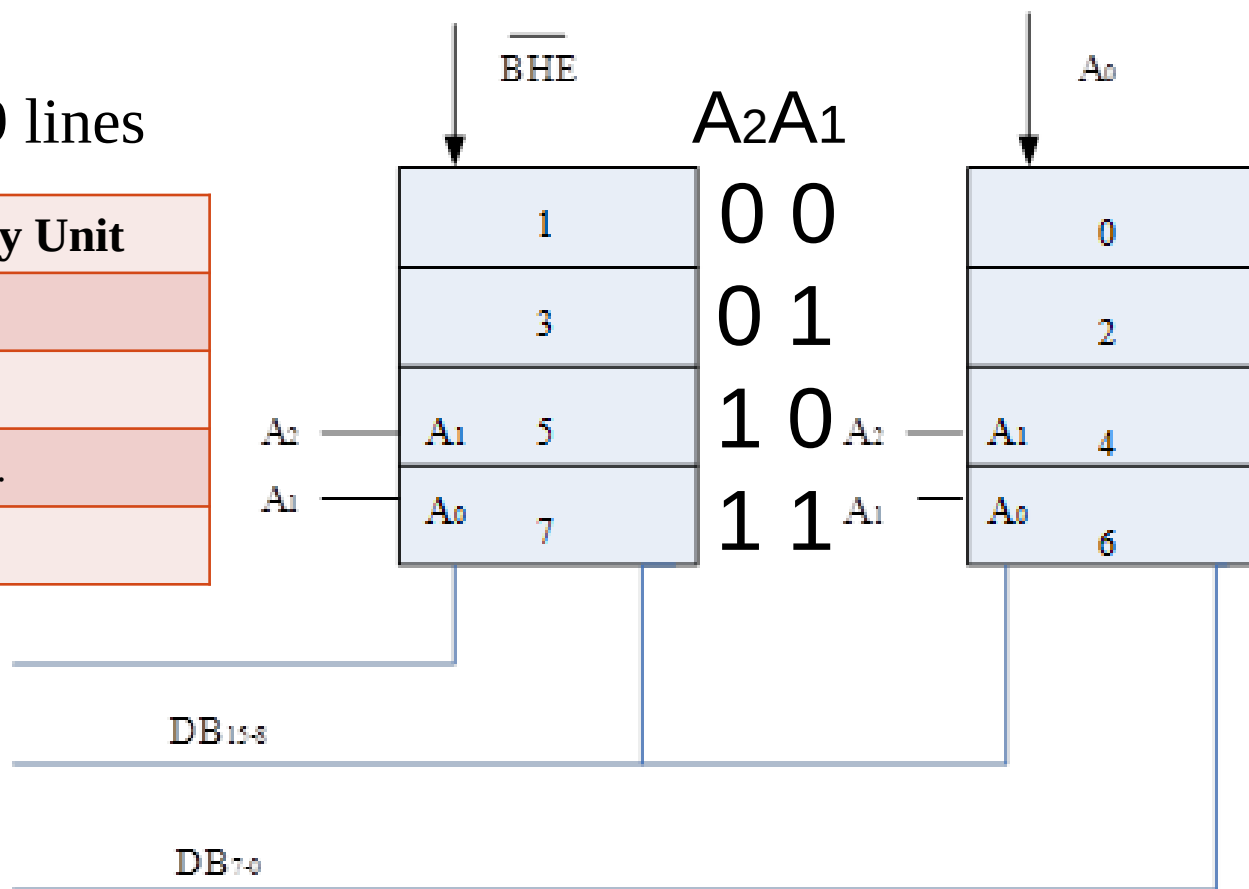
- A1-19 ~~~~ A0-18



# 8086存储器架构

- 寻址例子
  - 8 个单元
  - 3 个地址线
  - 512KB: 19 lines

| A <sub>2</sub> A <sub>1</sub> A <sub>0</sub> | Memory Unit |
|----------------------------------------------|-------------|
| 000                                          | 0#          |
| 001                                          | 1#          |
| ...                                          | ...         |
| 111                                          | 7#          |



# 存储器读写

- R/W 操作过程

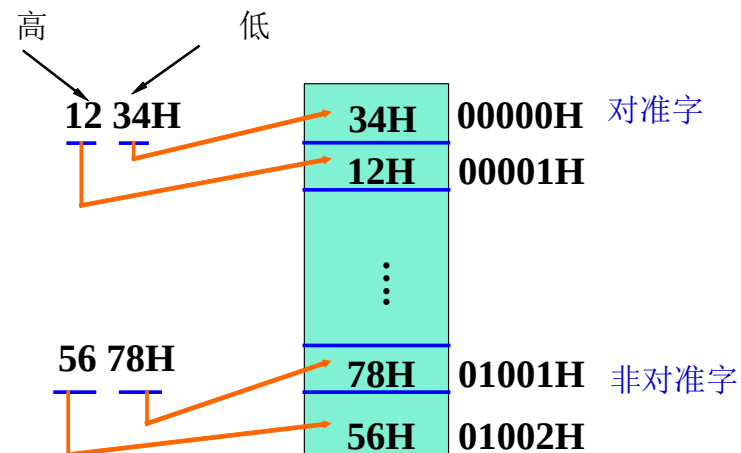
- 1字节(8 bits) 为基本单元
- R/W 8-bit 字节 或 16-bit 字
- 规则
  - 高字节对应高地址，低字节对应低地址
  - **Examples: 1234H (00000H) 5678H (01001H)**

- 字的地址

- 1个字占2个字节
- 低8位所在字节地址作为字的地址

- 对准字和非对准字 Word alignment and non-aligned word

- 奇地址
- 偶地址



读取效率不同

# 存储器读写

- 存储器读写

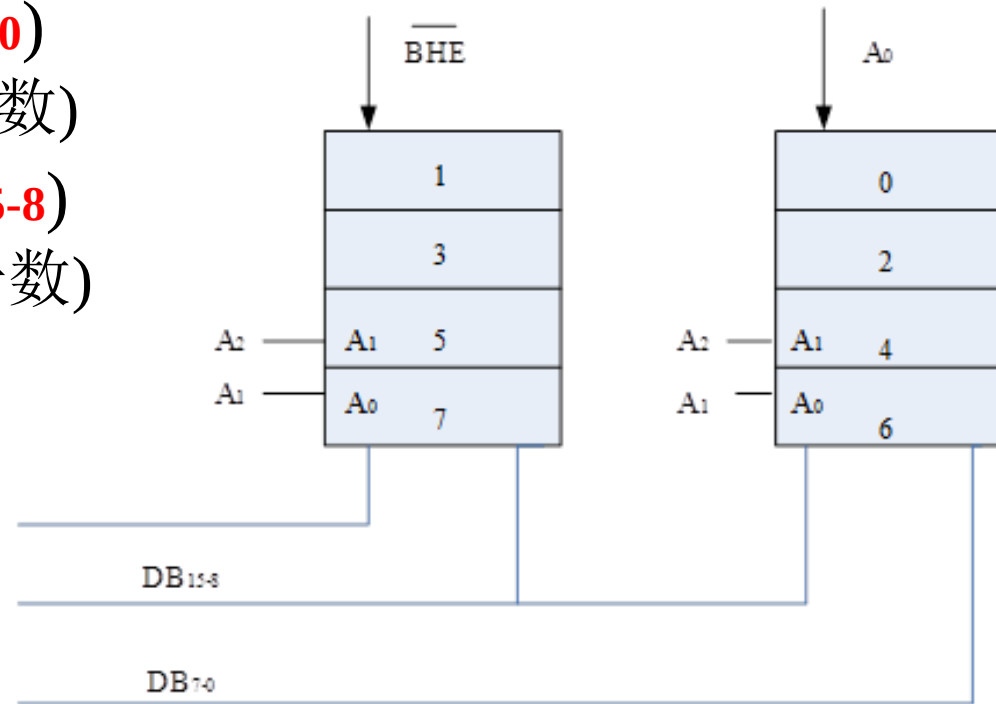
- 操作,  $\overline{\text{BHE}}$ ,  $\text{A}_0$ , 有效地址线, 举例

- 读写偶地址字节(DB7-0)

- `MOV AL, [SI]` SI (偶数)

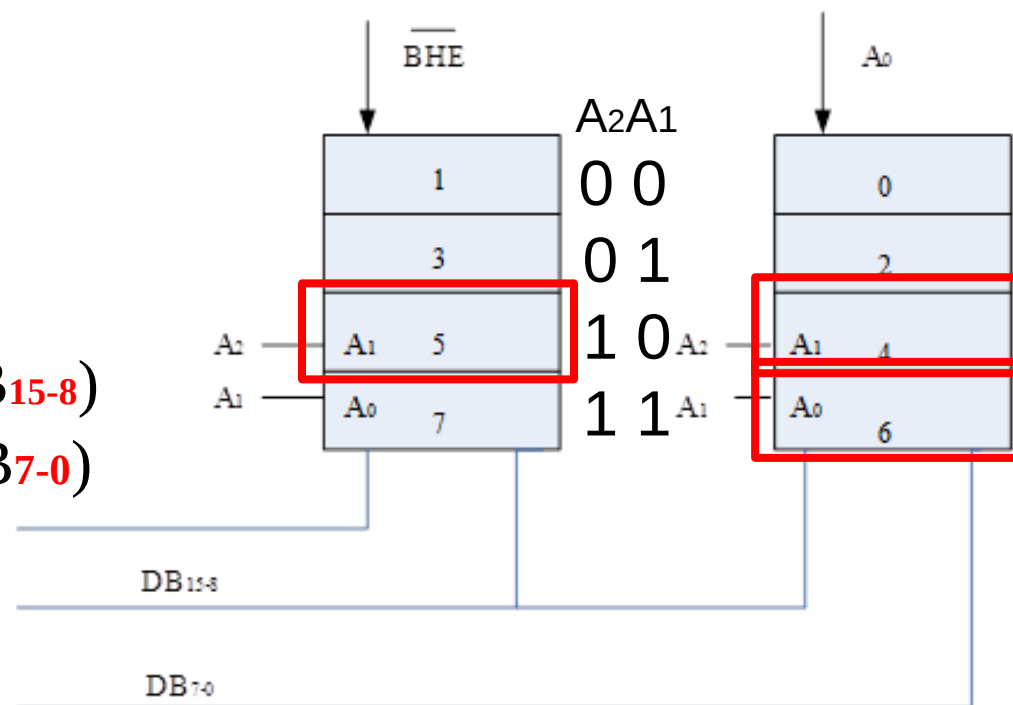
- 读写奇地址字节(DB15-8)

- `MOV [SI], AL` SI (奇数)



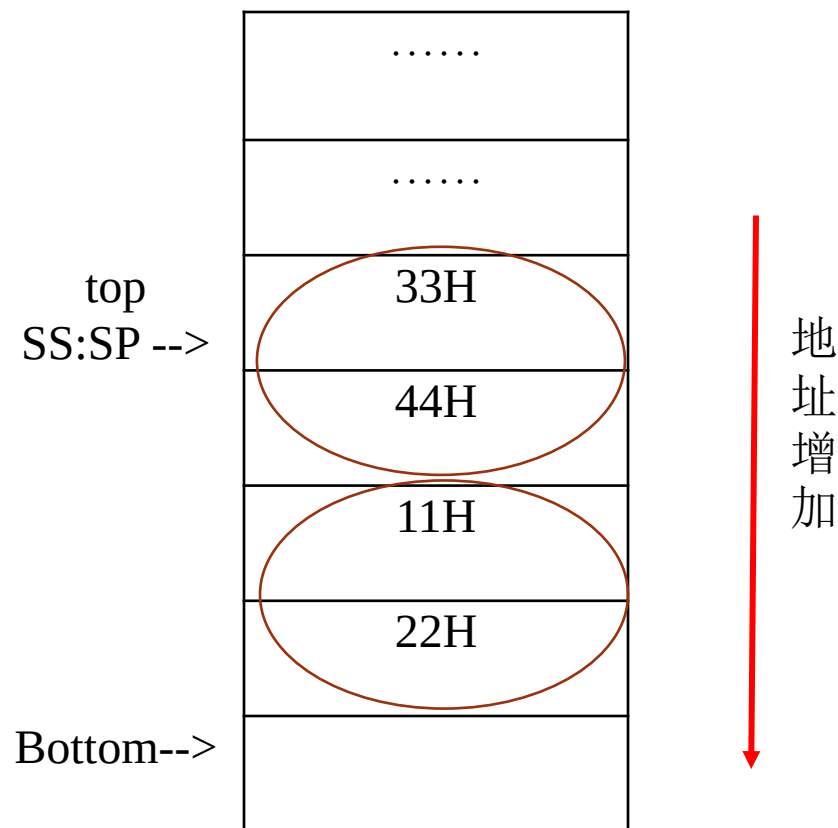
# 存储器读写

- 读写偶地址字(DB<sup>15-0</sup>)  
 $A_2A_1=10$   $\overline{BHE}=0$   $A_0=0$   
`MOV AX, [SI]` SI (偶数)
- 读写奇地址字  
 $A_2A_1=10$  ( $\overline{BHE}=0$   $A_0=1$ ) (DB<sup>15-8</sup>)  
 $A_2A_1=11$  ( $\overline{BHE}=1$   $A_0=0$ ) (DB<sup>7-0</sup>)  
(提供两次地址信息)  
`MOV [SI], AX` SI (奇数)
- Bus cycle总线周期
  - 占用2个总线周期: 读写非对准字
  - 2倍时间



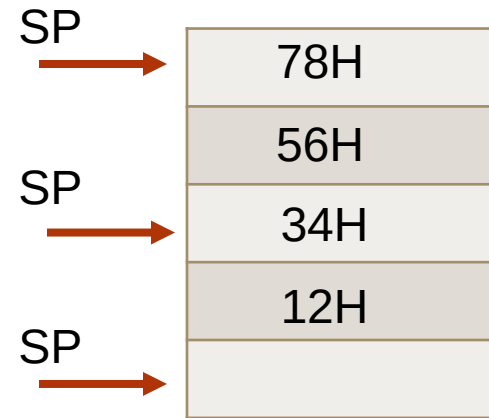
# 堆栈段Stack segment

- 堆栈Stack
  - 内存中特殊区域：FILO先入后出
- 示意图
  - 栈顶: 低地址
    - SS:SP
  - 栈底: 高地址
  - 入栈Push, 出栈pop操作
  - 按字进行操作
  - 应用: 程序调用与返回



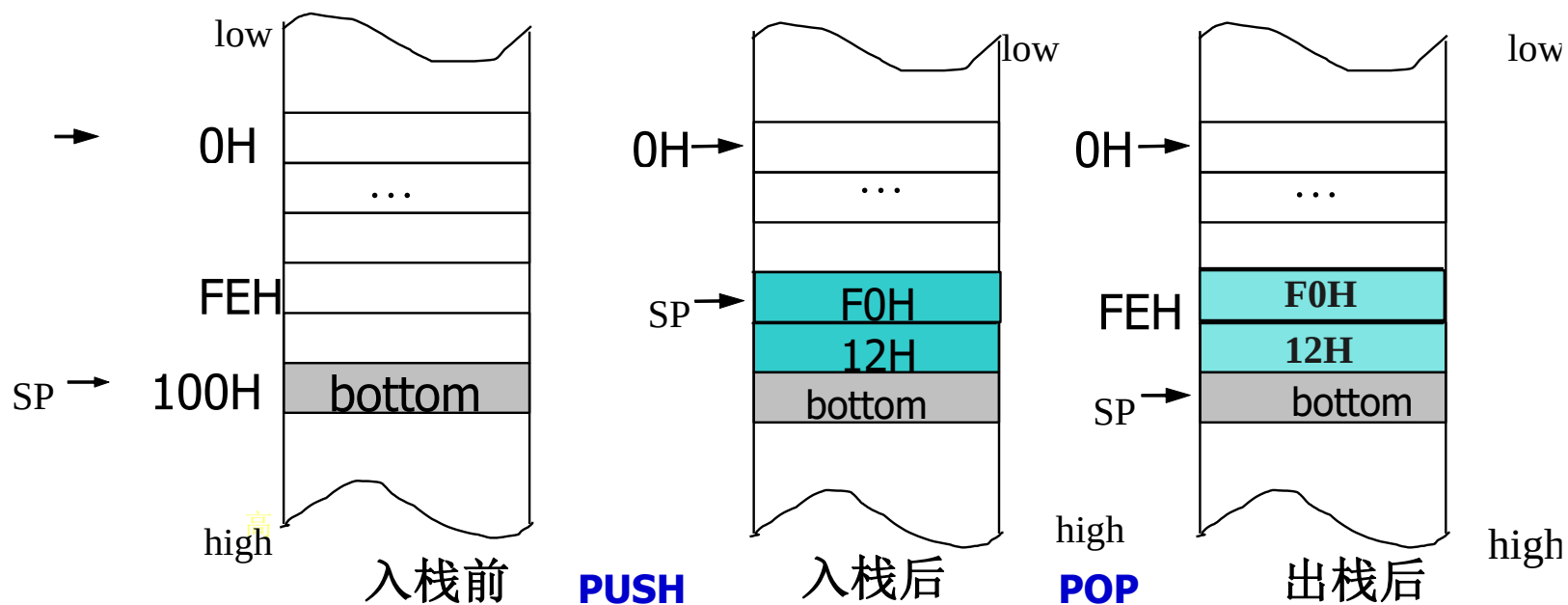
# 堆栈操作Stack operation

- PUSH
  - $SP = SP - 2$ , R/M数据(字) $\rightarrow [SS:SP]$
- POP
  - $[SS:SP] \rightarrow$  R/M数据(字),  $SP = SP + 2$
- **Examples**
  - $[SS:SP] = 5678H$   $SS = 100H$   $SP = 98H$   
POP BX
    - 出栈后  $SP = ?$   $BX = ?$
    - 占几个总线周期?



# 堆栈操作Stack operation

- 12F0H 入栈 / 出栈



- 初始SP=100H (空栈)
  - 堆栈容量? 最多存多少个字?



# 堆栈段Stack segment

- 使用时:
  - 用于处理中断返回和子程序调用返回
  - 临时保存数据
- **Examples**
- 注意
  - FILO先进后出
  - PUSH 和 POP 要成对使用

# Exercises

- 1. 初始  $SS=2360H$ ,  $SP=0800H$ , 该堆栈的物理地址范围? 若入栈10个字后  $SP=?$

$$23600H + (0 \sim 7FFH) = 23600H \sim 23DFFH$$
$$800H - 14H = 7ECH$$

- 2. 数据段物理地址范围:  $B4000H \sim C3FFFH$ ,  $DS=?$  偏移地址范围?

$$B400H \quad 0 \sim FFFFH$$

- 3. 若  $DS=7F06H$ , 从  $0075H$  连续存储6个字节:  $11H$ ,  $22H$ ,  $33H$ ,  $44H$ ,  $55H$  和  $66H$ 。节省时间如何读取? 节省指令如何读取?

$$7F060H + (75H \sim 7AH) = 7F0D5H \sim 7F0DAH$$



|        |       |
|--------|-------|
| .....  | ..... |
| 7F0D5H | 11H   |
| 7F0D6H | 22H   |
| 7F0D7H | 33H   |
| 7F0D8H | 44H   |
| 7F0D9H | 55H   |
| 7F0DAH | 66H   |

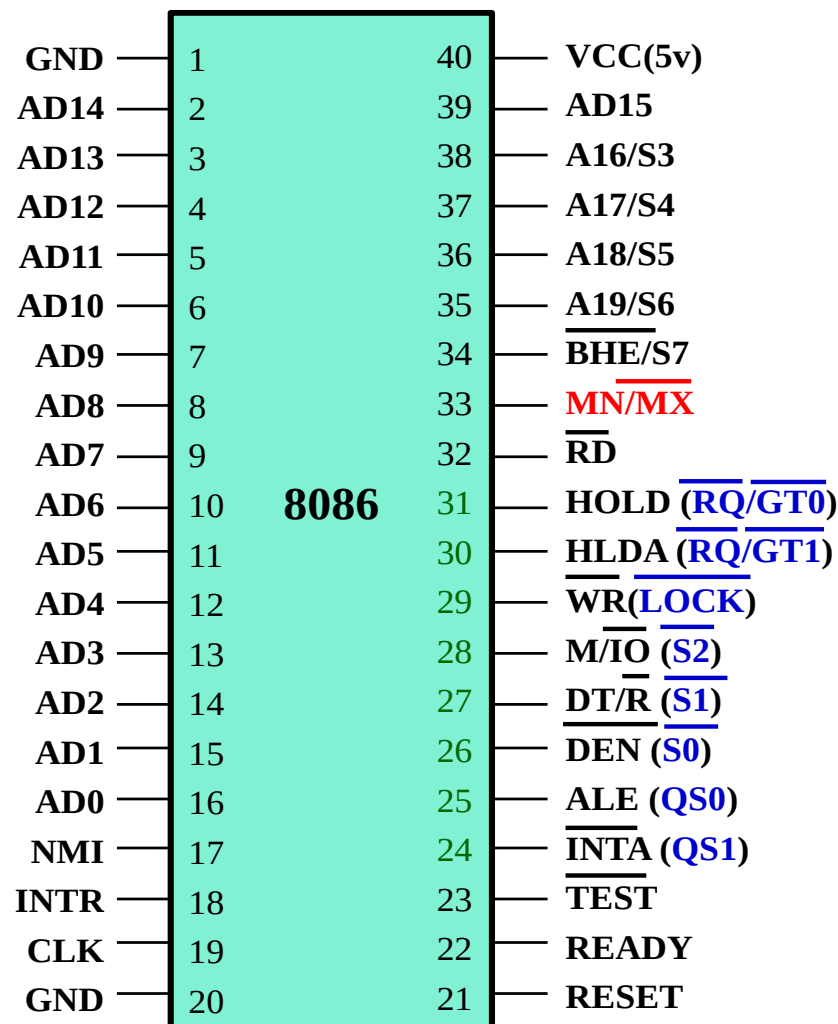
# Exercises



## 2.4 8086系统配置与时序图

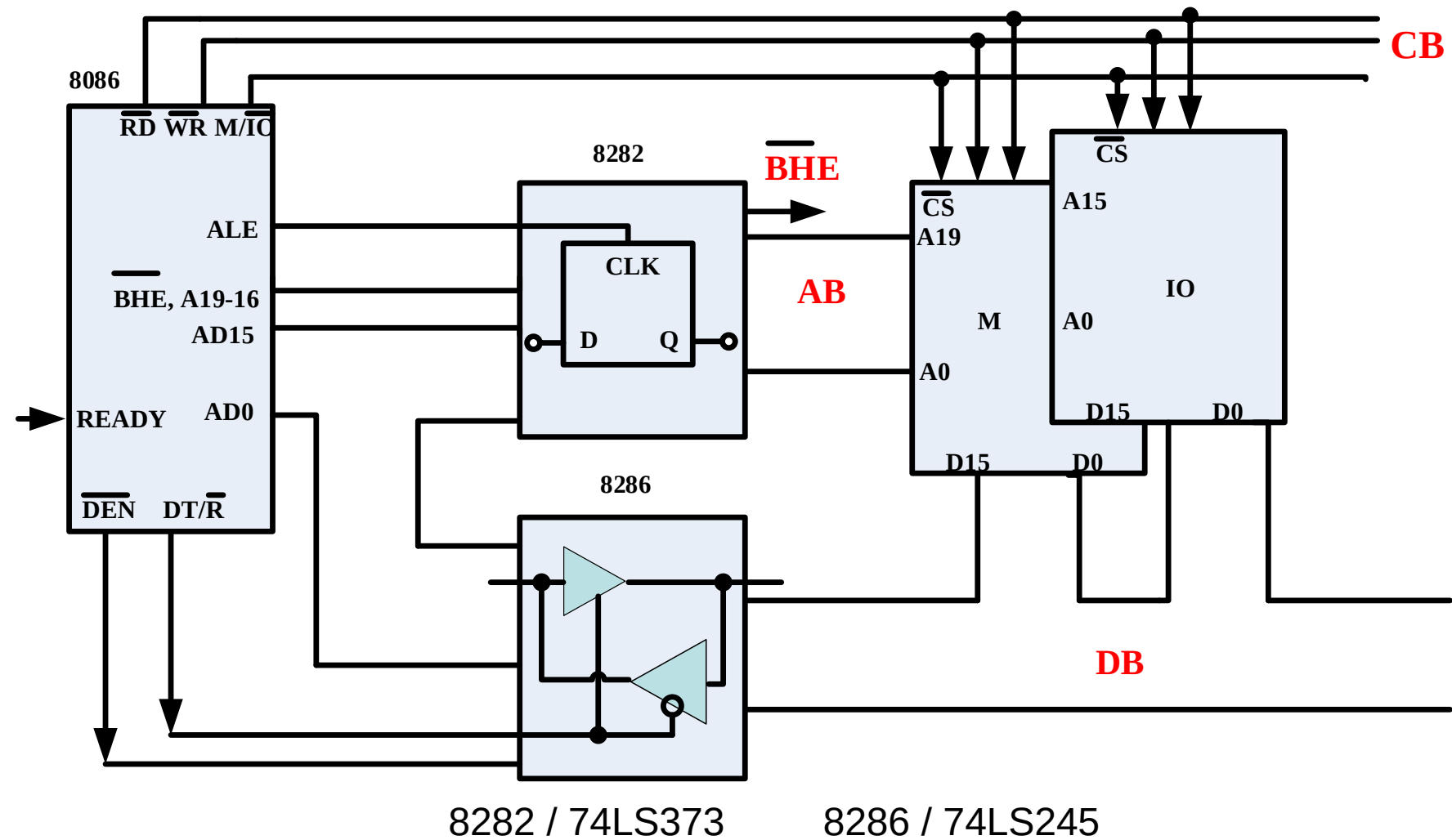
# 8086 CPU 系统配置

- 最小最大模式
  - $\overline{MN}/\overline{MX}$
- Minimum mode
  - 单处理器系统, 提供所有控制信号
- Maximum mode
  - 多处理器系统, 总线控制器
- 管脚定义不同24-31



minimum mode (maximum mode)

# 最小系统配置



# 最小系统配置



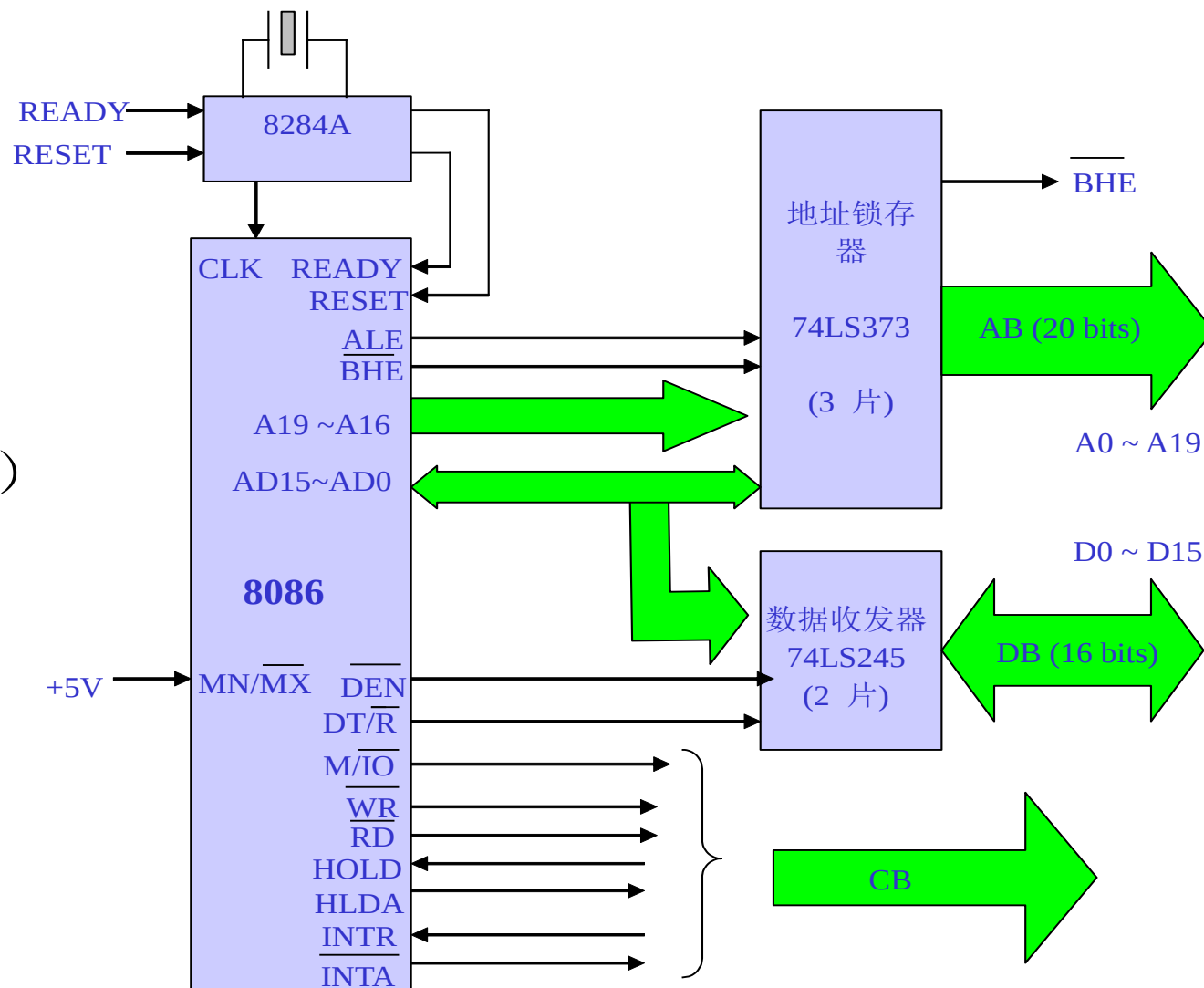
74LS373  
地址锁存器



74LS245  
双向数据收发器

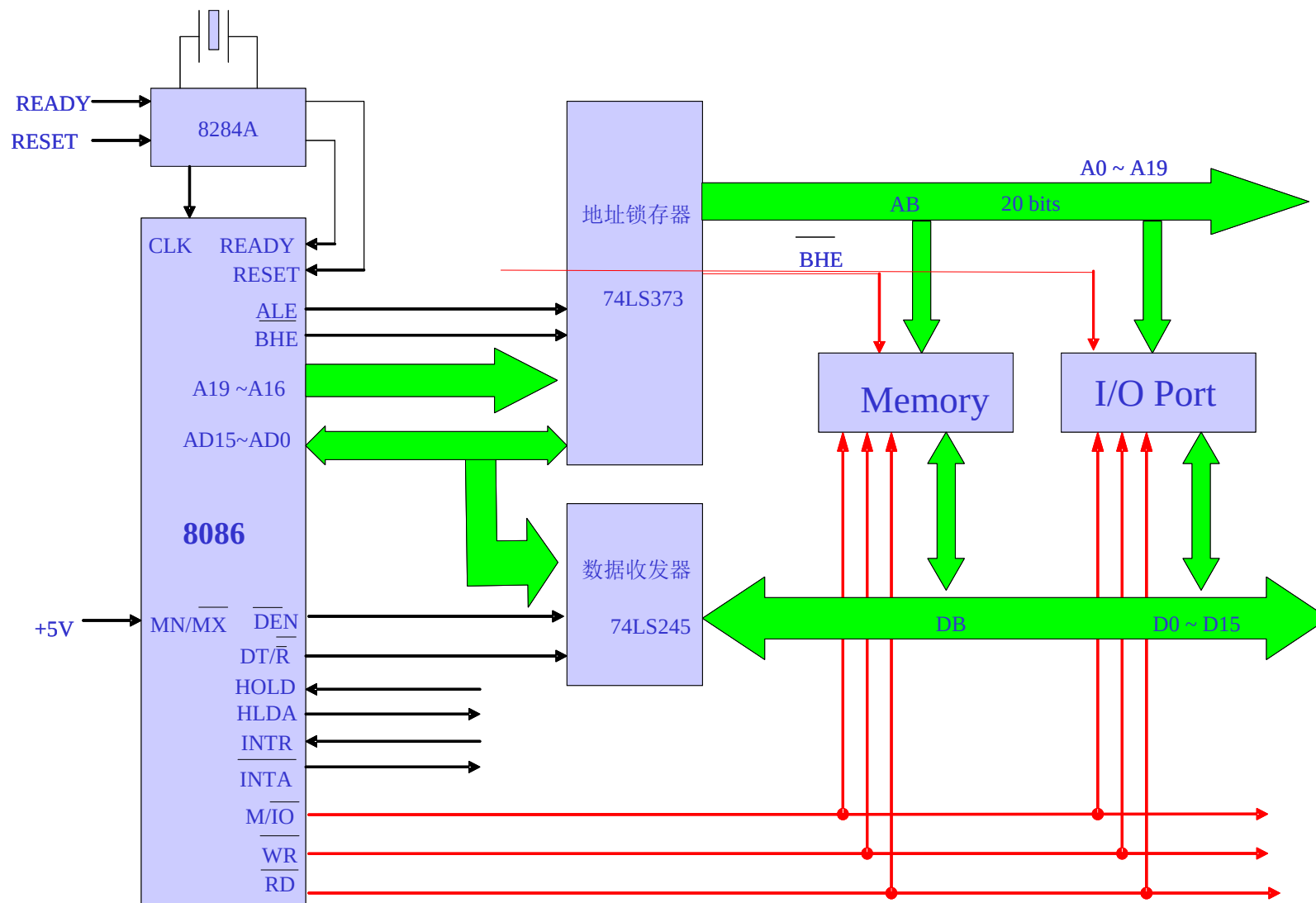
# 总线： AB DB CB

- $\overline{MN}/\overline{MX}$
- 8284A—时钟发生器
- 地址锁存器（1片锁存8个信号）  
(? 片)
- 双向数据收发器（1片收发8个信号）  
(? 片)





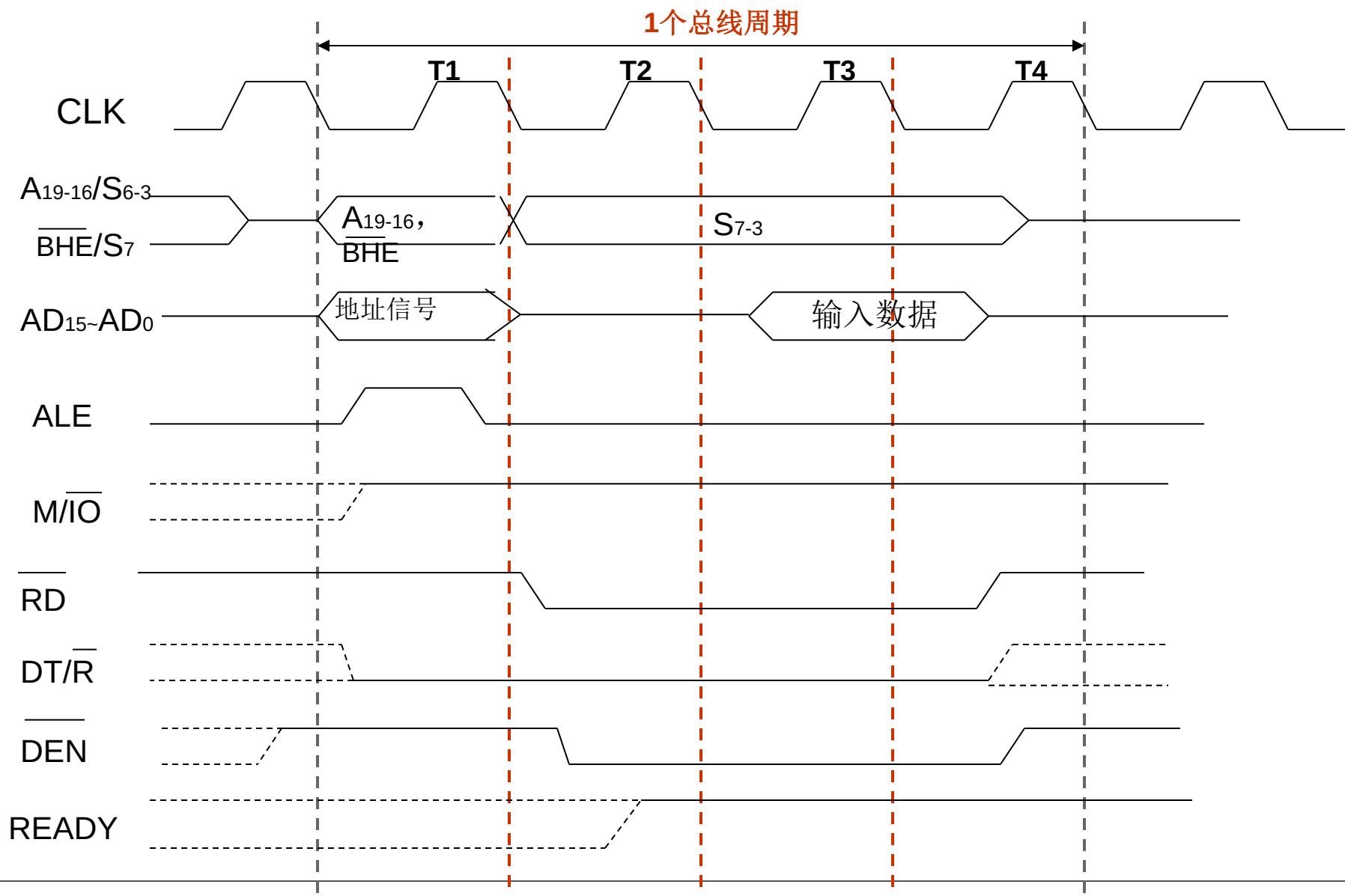
# 总线功能



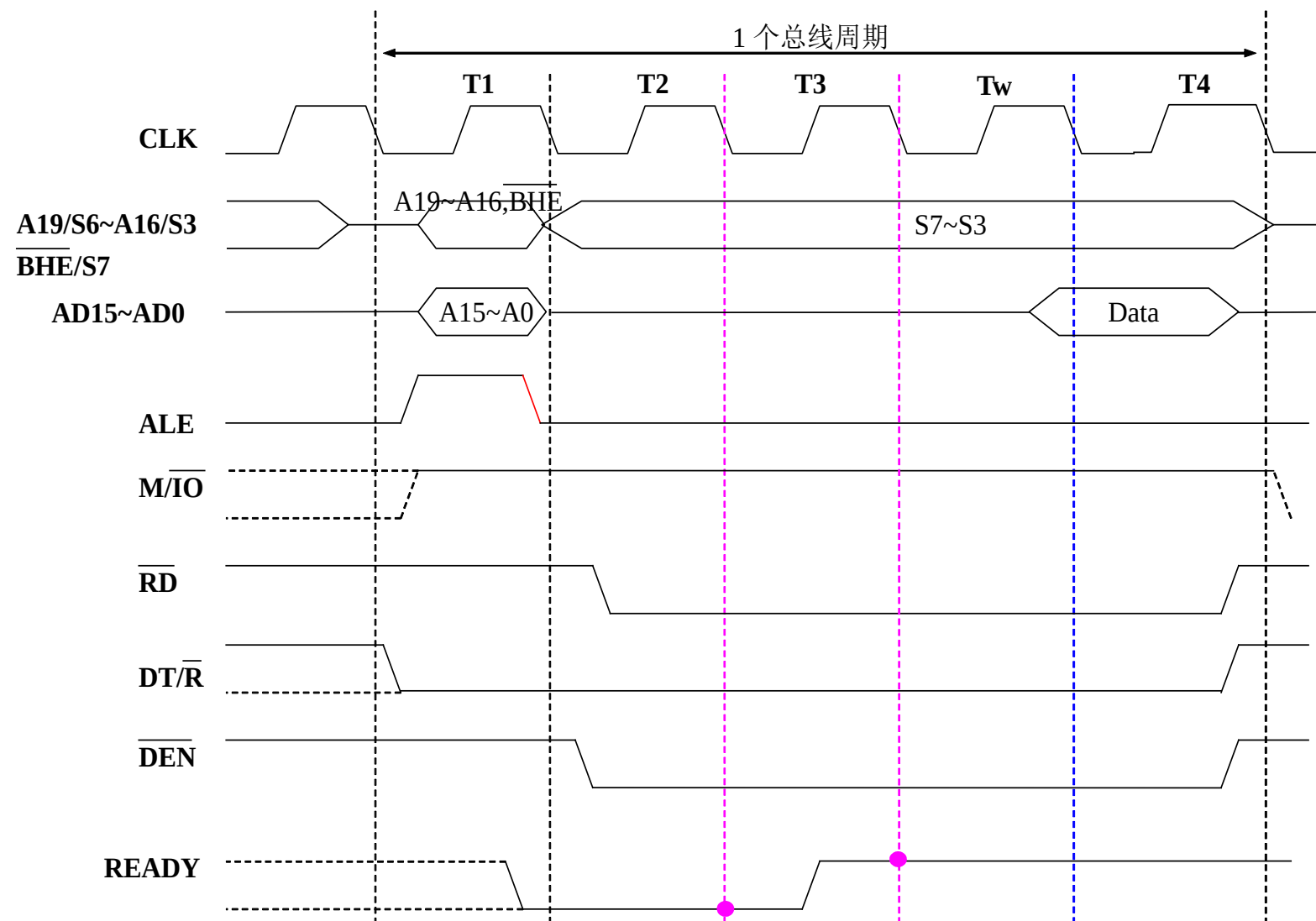
# 基本概念

- 时钟周期(T status)
  - 8086 CPU 时钟频率5MHz, 时钟周期(T status) 是 200ns
- 总线周期
  - 包含多个时钟周期(标准: 4 clock cycles; 可插入)
  - M/IO 读/写周期
- 指令周期Instruction cycle
  - 可以包含总线周期 (MOV AL, [100H])
  - 也可以不包含总线周期 (MOV AL, BL)
- 时序图
  - 执行不同指令时CPU管脚信号的变化

# 存储器读总线周期



# 插入 $T_w$



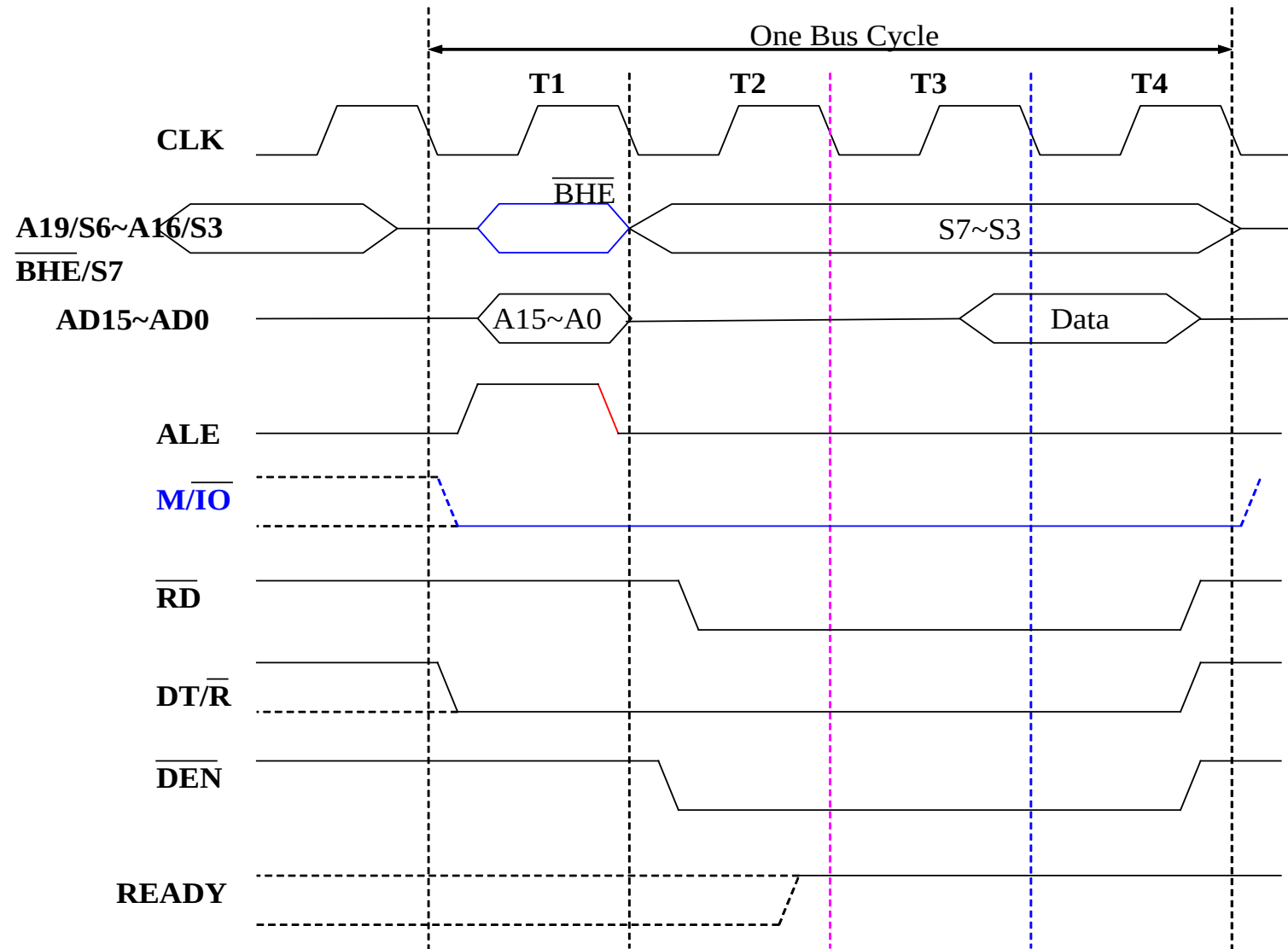
# Exercises



# I/O 读总线周期

- 与存储器读周期类似
- 2 个不同:
  - $M/\overline{IO}$
  - A19~A16 未用到

# I/O读总线周期

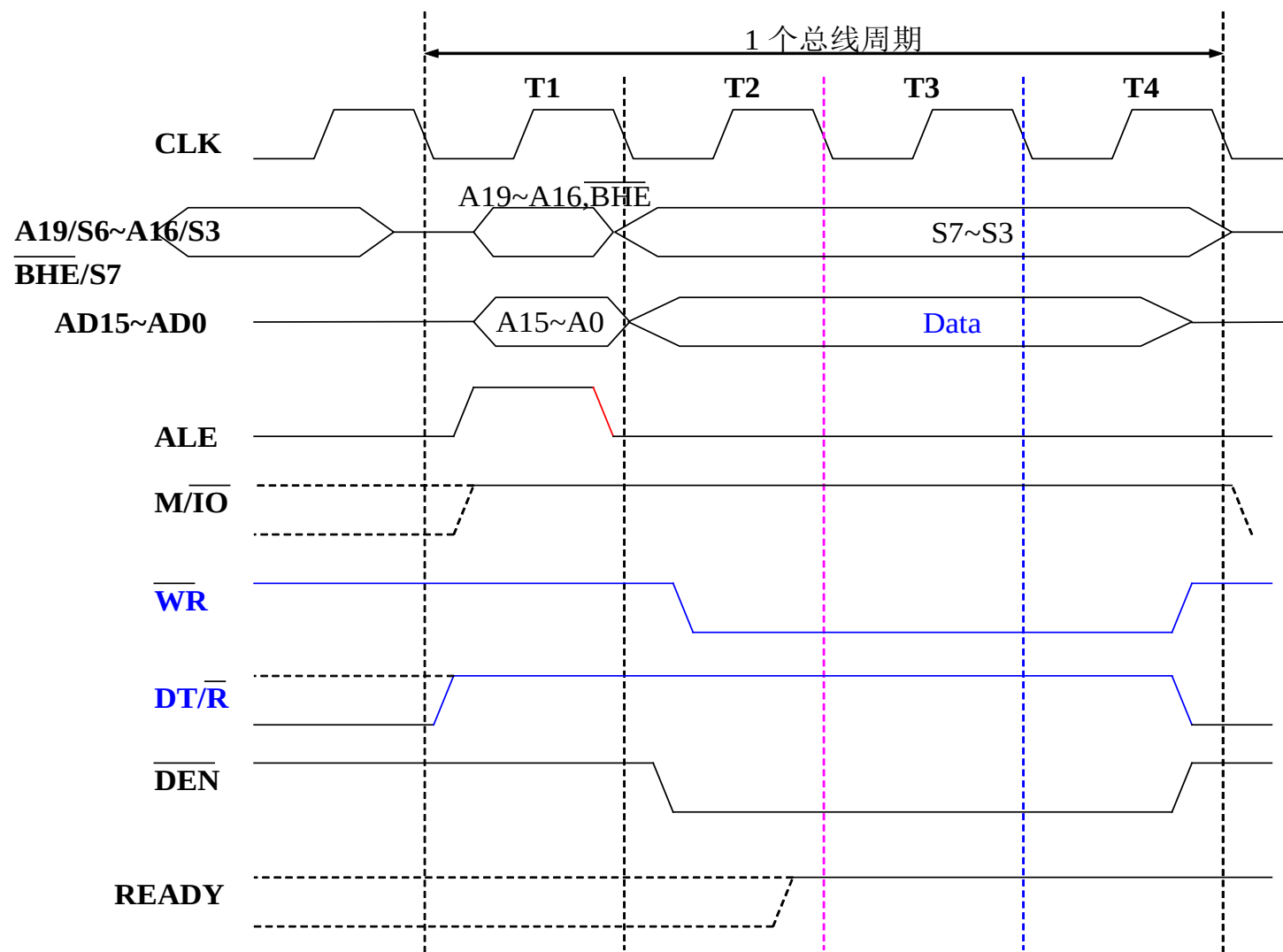


# 存储器写总线周期

- 3处与存储器读周期不同:
  - $\overline{DT}/\overline{R}$
  - A/D 复用信号:无高阻态
  - $\overline{WR}/\overline{RD}$



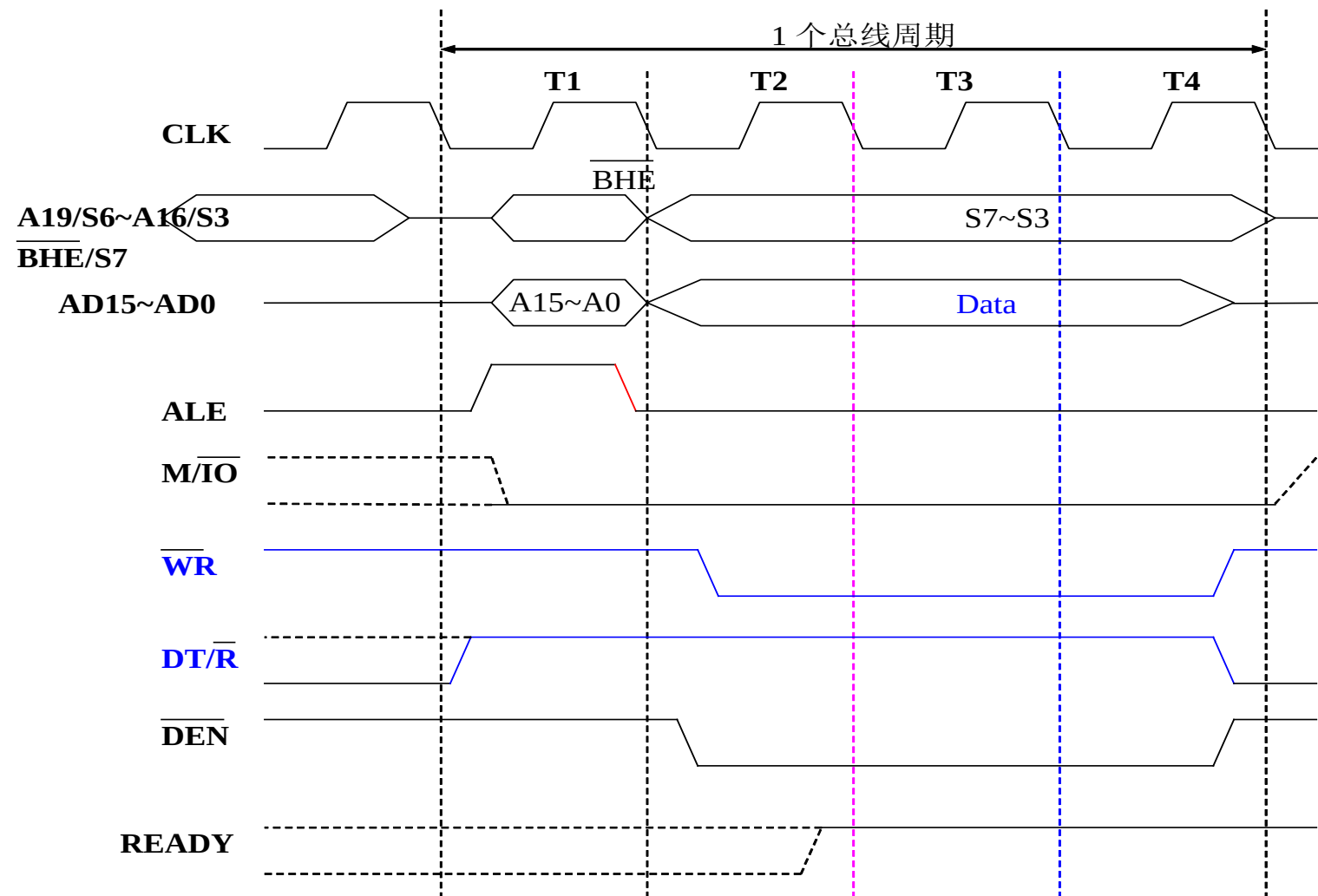
# 存储器写总线周期



# IO写总线周期

- 与存储器写周期类似
- 注意:
  - M/ $\overline{\text{IO}}$
  - A19~A16 未用到
- 与IO读周期有3处不同:
  - DT/ $\overline{\text{R}}$
  - $\overline{\text{A/D}}$  复用信号: 无高阻态
  - $\overline{\text{WR/RD}}$

# IO写总线周期



# 第二章复习

- 8086 CPU
  - EU and BIU, ALU, FR (PSW), 通用寄存器, 地址加法器, 指针和变址寄存器, 段寄存器, IP, 指令队列, CPU Reset
- 8086 存储器
  - 存储器分段, 物理地址, 逻辑地址, 段地址和偏移地址, 缺省搭配关系, 对照, 字和字节存储与读写处理, 8086奇偶存储体 8086系统配置与时序
- 8086 CPU 最小系统配置, 地址数据信号复用, 地址锁存与地址总线, 双向数据收发器与数据总线
- Memory/IO 读写周期, 时序图

# Discussion

- How to learn a new chip/part?
  - 内部结构& 工作原理
  - 管脚
  - 与其他芯片连接（系统配置）
  - 工作模式和时序
  - 编程Programing

# 作业

- 1. 两个带符号数 1011 0100B 和 1100 0111B 相加，运算后各标志位的值等于多少？哪些标志位是有意义的？如果把这两个数当成无符号数，相加后哪些标志位是有意义的？
- 2. 段基地址装入如下数值，则每段的起始地址和结束地址分别是什么？  
(1) 1200H  
(2) 3F05H  
(3) OFFEH
- 3. 已知 CS:IP=3456:0210H，CPU 要执行的下条指令的物理地址是什么？
- 4. SS:SP=2000:0300H，则堆栈在内存中的物理地址范围是什么？执行两条 PUSH 指令后，SS:SP=? 再执行一条 PUSH 指令后，SS:SP=?